

RISC-V Supervisor Domains Access Protection

RISC-V SmMTT Task Group

Version v0.49, 2026-01-15: Draft

Table of Contents

Preamble . . . 1
Copyright and license information . . . 2
Contributors . . . 3
Glossary . . . 4
1. Introduction . . . 6
1.1. Motivation and Goals . . . 6
1.2. Memory Isolation - Theory of operation (informative) . . . 7
2. Summary of extensions for Supervisor Domain Access Protection . . . 11
3. Smsdid: Supervisor Domain Identifier and Protection Register . . . 12
3.1. Machine-level Memory Protection Tables (mmpt) . . . 12
3.2. Machine Memory Protection Fence Instruction . . . 13
3.3. M-mode Supervisor Domain Fine-Grain Invalidation Instruction . . . 14
3.4. Extension to Behavior of Memory-Management Fences . . . 15
3.5. Machine supervisor domain configuration (msdcfg) . . . 15
4. Machine-mode Memory Protection Table (MPT) . . . 17
4.1. Smmpt34: Page-Based Memory Protection System . . . 17
4.1.1. MPT access type permissions lookup process . . . 20
4.2. Smmpt43: Page-Based 43-bit Memory Protection System . . . 20
4.3. Smmpt52: Page-Based 52-bit Memory Protection System . . . 22
4.4. Smmpt64: Page-Based 64-bit Memory Protection System . . . 23
5. I/O MPT Checker . . . 24
5.1. Placement of I/O MPT Checker . . . 24
5.2. Handling I/O MPT Checker violations . . . 27
5.3. I/O MPT Checker Register Interface . . . 28
5.4. Capabilities (capabilities) . . . 29
5.5. Status register (status) . . . 29
5.6. Control (control) . . . 30
5.7. Command register (command) . . . 31
5.7.1. Configure a SDCL rule - SET_SDCL_ENTRY . . . 32
5.7.2. Read the configuration of an SDCL rule - GET_SDCL_ENTRY . . . 35
5.7.3. Set Supervisor Domain Configurations - SET_SDCFG_ENTRY . . . 35
5.7.4. Get Supervisor Domain Configurations - GET_SDCFG_ENTRY . . . 38
5.7.5. MPT Permission Cache Invalidation - MPTINVAL . . . 38
5.7.6. I/O Fence Operation - IOFENCE . . . 39
5.8. Treatment of Device-Side Address Translation Caches . . . 40
6. Supervisor Domain Interrupt Assignment . . . 42
6.1. IMSIC Extension for Supervisor Domains . . . 42
6.2. APLIC Extension for Supervisor Domains . . . 43

6.3. "Smsdia" : Standard Extension for Supervisor Domain Interrupt Assignment 44
6.3.1. Supervisor Interrupt Domain Number 44
6.3.2. Hypervisor Guest External Interrupt Registers (hgeip and hgeie) 45
6.3.3. Machine supervisor interrupt domain external interrupt pending (msideip/msideiph) . . 45
6.3.4. Machine supervisor interrupt domain external interrupt enable (msideie/msideieh) . . . 46
6.3.5. Machine and Supervisor Interrupt registers (mip/mie and sip/sie) 46
6.4. "Smgeien/Ssgeien" : Standard Extension for Machine Guest External Interrupt Enables 47
7. Supervisor Domain External Trace and Debug Allow Controls 50
7.1. Smsdedbga: External Debug allowed control for Supervisor Domain 50
7.2. Smsdetrca: External Trace allowed control for Supervisor Domain 51
8. Capacity and Bandwidth QoS Register Interface (CBQRI) for Supervisor Domains 52
8.1. Introduction to CBQRI and Ssqosid 52
8.2. Security objective 52
8.3. Smqosid - Machine-Mode Quality-of-Service Identifiers 53
8.4. Smsdqosid - Supervisor Domain Quality-of-Service Identifiers 53
8.5. Supervisor Domain QoS Register interfaces (QRI) 56
Bibliography 60

Preamble



This document is in the Development state

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2024 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order): Alvin Che-Chia Chang, Andy Chiu, Andy Dellow, Anup Patel, Dean Liberty, Deepak Gupta, Eckhard Delfs, Gregor Haas, Guerney Hunt, Jack Andrew, Krste Asanovic, Mark Hill, Nick Wood, Osman Koyuncu, Paul Elliott, Ricardo Ramirez, Ravi Sahita (Editor), Samuel Holland, Samuel Ortiz, Ved Shanbhogue, Wojchiech Ozga

Glossary

ABI	Application binary interface (ABI).
AIA	RISC-V Advanced Interrupt Architecture (AIA) [1] interrupts.
AP	Application processors (AP)s can support commodity operating systems, hypervisors/VMMs and applications software workloads. The AP subsystem may contain several processing units, on-chip caches, and other controllers for interfacing with memory, accelerators, and other fixed-function logic. Multiple APs may be used within a logical system.
Attestation	The process by which a relying party can assess the trustworthiness of the confidential computing environment based on verifying a set of evidences that are cryptographically endorsed by a hardware root-of-trust.
Confidential Computing	A computing paradigm that protects data-in-use by performing computation in a hardware-based, attested, execution environment.
CoVE	**Co**nfidential **V**M **E**xtension is the set of RISC-V ABI extensions defined in [2] that enables confidential computing for hardware virtual machines (VMs) on RISC-V platforms.
HW RoT	Hardware (HW) Root of trust (RoT) is the isolated hardware/software subsystem with an immutable ROM firmware and isolated compute and memory elements that form the Trusted Compute Base (TCB) of a TEE system. The RoT manages cryptographic keys and other security critical functions such as system lifecycle and debug authorization. The RoT provides trusted services to other software, for which it is the TCB, such as verified boot, key management, security lifecycle management, sealed storage, device management, crypto services,attestation etc. The RoT may be an integrated or discrete element, and may be used to manage device identies for attestation.
Hypervisor	Software running in HS-mode that manages virtual machines (VMs) by virtualizing hart, guest physical memory and input/output (IO) resources.
IMSIC	Incoming Message-signaled Interrupt Controller (IMSIC).
MMIO	Memory mapped I/O (MMIO).
MMU	Memory management unit (MMU).
MPT	Machine-level Memory Protection Tables (MPT).
Relying party	An entity that uses the attestation process to assess the trustworthiness of an attester.

Supervisor Domain (SD)	A RISC-V privileged architecture extension defined in this specification, to support isolation across more than one supervisor execution context. Supervisor domains enable the reduction of the supervisor Trusted Computing Base (TCB), with differentiated access to memory and other platform resources. All host software elements including OS and type-1 or type-2 VMM and hosted VMs operate in a "hosting" supervisor domain. The hosting supervisor domain may interact with multiple distrusting supervisor domains via the support of a root domain security manager. The alternate supervisor domains may each host their own software and applications.
TCB; Also, System/Platform TCB	Trusted computing base (TCB) is the hardware, software, and firmware elements that are trusted by a relying party to protect the confidentiality and integrity of the relying parties' workload data and execution against a defined adversary model. In a system with separate processing elements within a package on a socket, the TCB boundary is the package. In a multi-socket system the Hardware TCB extends across the socket-to-socket interface, and is managed as one system TCB. The software TCB may also extend across multiple sockets.
TEE	Trusted execution environment (TEE) is a set of hardware and software mechanisms that allow creating attestable and isolated execution environment.
Tenant workload	All software elements owned and deployed by a single organization that may be hosted by a platform operator e.g. cloud provider on a platform that can host more than one organizations workload simultaneously. For example, in a virtualized environment, the tenant workload elements may include VS-mode guest kernel and VU-mode guest user-space applications. Tenant workloads may also operate in the context of one of more supervisor domains.
VM	An efficient, isolated duplicate of a real computer system. In this specification it refers to the collection of resources and state that is accessible when a RISC-V hart supporting the hypervisor extension executes with the virtualization mode set to 1.
VMM	Virtual Machine Monitor. Also referred to as hypervisor.

1. Introduction

1.1. Motivation and Goals

RISC-V privileged architecture [3] defines execution mode for supervisor software called S-mode. S-mode software may optionally enable Hypervisor extension to host virtual machines. Typically, there is a single supervisor domain of execution with access to all physical memory. This document describes **Supervisor Domain Access Protection** - a RISC-V privileged architecture extension to support physical address space (memory and devices) isolation for more than one supervisor domain. Supervisor domains enable trusted execution use cases for RISC-V platforms. Supervisor domains may also be used to reduce the supervisor Trusted Computing Base (TCB), with differential access to memory and other platform resources e.g. in Confidential Computing, TEE Security Services, Secure Devices etc.

Tenant (application or VM) workloads on multi-tenant platforms rely on hardware-based isolation primitives that are managed by the host/privileged software. Traditional host software (operating systems and virtual machine monitors) have unfettered access to system memory, devices and hardware isolation mechanisms such as the memory management unit (MMU) or physical memory protection (PMP) registers. Typical multi-tenant platforms also have a very large Trusted Computing Base (TCB) and have many threat actors that influence the TCB. For example, in a multi-tenant data-center environment, the host software, devices and device drivers, cloud operators, orchestration services, other tenant workloads etc. are all considered threat vectors. Additional threats originate due to deployment models, for example, an embedded platform deployed in the field is exposed to physical threat vectors. Hence, in many scenarios isolated supervisor domains are desired to be able to express differentiated trust models and secure access to platform resources. A supervisor domain uses resources (such as memory/IO regions, processing elements, devices and interrupts) to perform their function.

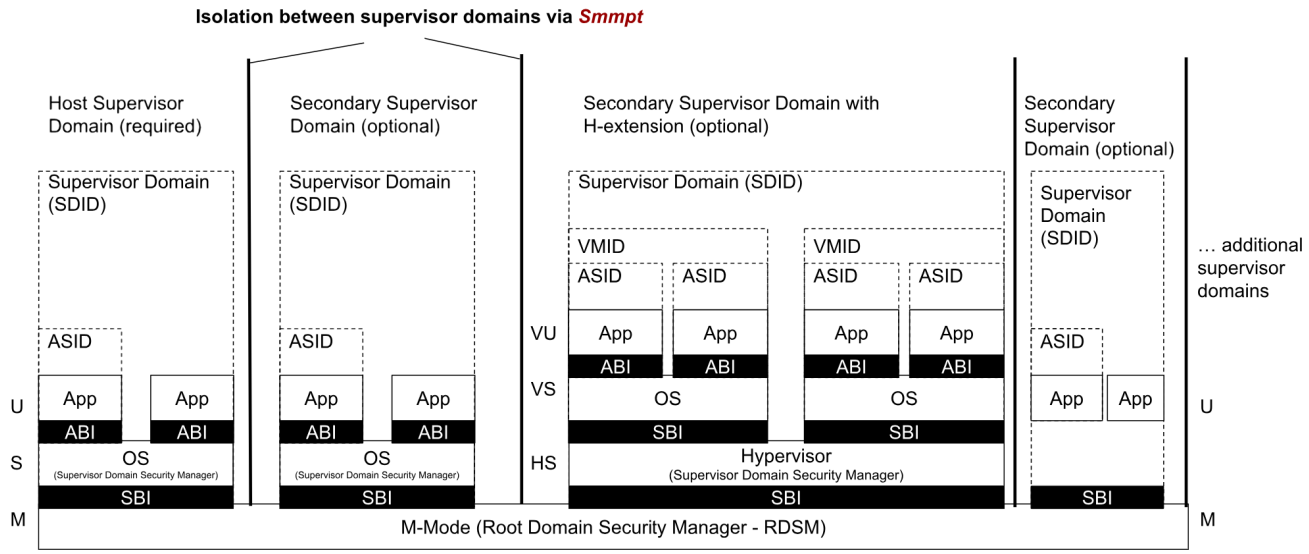


Figure 1: Supervisor Domains

A supervisor domain is associated with a set of physical address regions that are isolated from other supervisor domains on the same platform, with only the Root Domain Security Manager (RDSM) with access to all of the physical address space. A supervisor domain identifier (SDID) is

associated with the hart operating in the context of a supervisor domain to facilitate physical address protection fences on a per supervisor domain basis. Supervisor domains must rely on a TCB which consists of the RDSM (software) and hardware (hart, SoC, Root-of-trust) that enforces the isolation properties for the supervisor domain. The RDSM may utilize PMP/ `Smepmp` and/or the `Smmpt` (Machine-level Memory Protection Tables) extension described in this specification to isolate physical memory between supervisor domains. Isolation of the workloads within a supervisor domain is the responsibility of the OS/hypervisor managing the supervisor domain, here referred to as the Supervisor Domain Security Manager (SDSM).

A key goal of using multiple domains is to be able to reduce the common TCB across domains, and should enable the attestation [4] of each domain independently from other domains. Sensitive data may be entrusted to a particular domain after verifying the trust properties statically (via boot) or dynamically (via attestation). These trust properties are established as part of the hardware and software supply chain, system configuration and may be additionally evaluated using attestation mechanisms. The security certification of the RDSM is desirable but out of scope of this specification.

Use cases for supervisor domain isolation range from embedded to application/server-class platforms. Some examples where supervisor domains can be used are:

- A trusted execution environment domain that isolates security services/applications.
- A confidential computing [4] domain which enforces confidentiality and integrity for workload data-in-use from the host/untrusted hypervisor, along with attestation of the TCB.
- A host (operator) domain that manages resources on the platform, and may assign resources to other domains.
- A service-provider domain that has exclusive access to some devices.

In order to avoid re-factoring of deployed software, workloads and applications, new hardware primitives are required to support flexible isolation of data in caches and memory. The new primitives are also required to isolate resources such as interrupts, IO, QoS mechanisms and debug/trace mechanisms for robust isolation of supervisor domains. The hardware primitives must support performant and scalable physical memory isolation at a page-level to support rich-OS memory management models. This specification describes the set of architecture extensions to support the requirements for supervisor domain isolation for RISC-V processor-based platforms.

1.2. Memory Isolation - Theory of operation (informative)

Supervisor Domain Access Protection extensions are used by M-mode RDSM to program access policies for supervisor domain operation. The `Smmpt` extension enables the RDSM to program permissions for physically-addressed memory (or device-mapped regions) by a hart/device operating within a supervisor domain. Associating a hart/device with a supervisor domain implies that any physical-addressable region access occurring in the context of the supervisor domain is subject to access-checks for that domain. Hence, software or hardware accesses that originate from supervisor domains other than the allowed supervisor domain can be explicitly prevented/allowed. The RDSM has access to physical memory for all supervisor domains. In typical security usages, write accesses to the MPT structures must be restricted and managed by the RDSM.

Memory regions may be accessed by harts or by other devices on the platform. When harts and devices are assigned to a supervisor domain, the hart/device is said to perform memory accesses in the context of that supervisor domain. For all accesses using a physical address, the SDID is the supervisor domain identifier programmed into a CSR. This CSR is programmed on the hart by the Root Domain Security Manager (RDSM). The assignment of the hart/device to a supervisor domain may be static (e.g. device assignment to a VM) or dynamic (e.g. scheduling a VM virtual cpu within a domain). The MPT for the supervisor domain active on the hart is programmed on the hart along with the supervisor domain identifier. The MPT does not perform any address translation; it simply provides access permissions for the physically addressed regions/pages to enforce the isolation properties per the use case requirements (see Figure 2 for a high-level illustration and Chapter 4 for normative details).

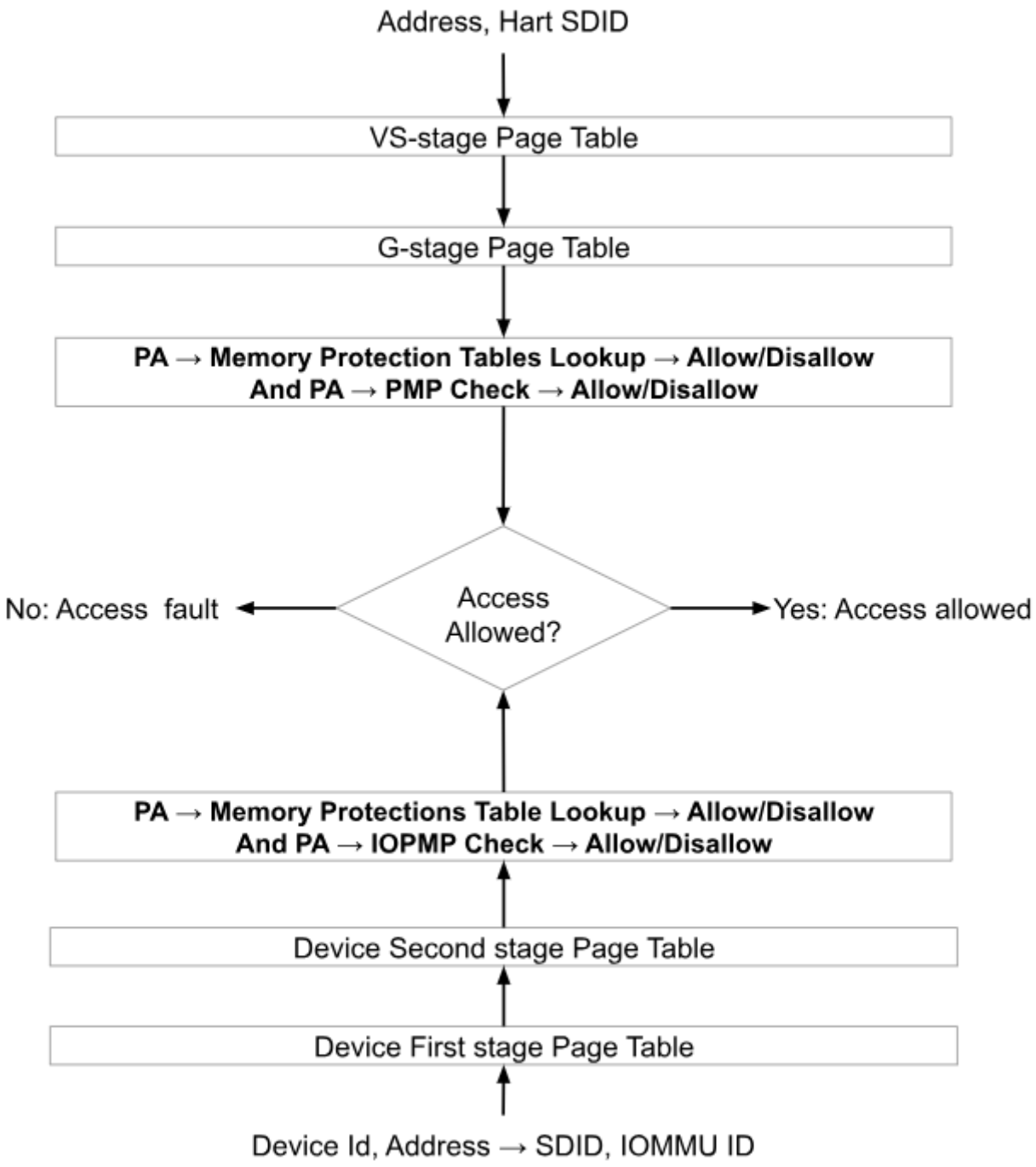


Figure 2: MPT lookup for Supervisor Domain Access

The MPT checker is a functional block that looks up the MPT using the physical address of the access as an index to retrieve the access permissions for the supervisor domain. This checker thus enforces that for a load initiated by the hart, the physical address is readable, and for a store initiated by the hart, the physical address is also writable, else reports a fault. An MPT access violation is always reported as a trap to the M-mode RDSM. The MPT checker may be implemented as an MMU extension in the hart, and/or in the IO interconnect to check device accesses. The MPT checker is designed to work together with the page-based virtual memory (MMU, IOMMU) systems and Physical Memory Protection (PMP, Smepmp, IOPMP) mechanisms. Read and Write permissions for memory are derived from the page table, the PMP and the MPT - an access is allowed only when all protection mechanisms allow the access.

MPT may be used to provide permissions for physical memory addresses that hold regular main memory or IO memory. Memory may be assigned to the RDSM to bootstrap the subsequent run-time lookup structures for MPT. All memory should be covered by the MPT, though some memory may not be eligible to be qualified for assignment to a specific supervisor domain. This limitation may arise due to platform configuration and security policies - for example, if the platform security policy requires memory for a domain to be encrypted and some memory access paths are not enforced via an inline memory encryption engine. It is expected that the RDSM can use trusted platform-specific methods to enumerate which regions can be designated as access-controlled via the MPT.

MPT must support both static and run-time configurability. A memory region (consisting of one or more pages) may be (re)assigned from one domain to another at run-time e.g. this is done by revoking the permission for one domain and assigning permissions to another domain. Run-time configuration may be performed via M-mode CSRs and/or in-memory structures. The in-memory structures used for MPT must themselves be access-limited to the RDSM by use of the MPT structures to disallow any supervisor domain from accessing the structures unless explicitly delegated by the Root Domain Security Manager (RDSM) to a particular domain (per use case policies). To support MPT dynamic reconfiguration, an interface is expected to be provided to set the attributes by passing requests to a trusted driver (in the RDSM) that can reconfigure the memory region assignment. Converting memory regions assignment from one domain to another might involve platform-specific operations based on the enforcement mechanism, such as TLB/cache flushes, that must be enforced by the RDSM and hardware. The RDSM is expected to change the settings and flush caches if necessary, so the system is only incoherent during the transition between domain assignment settings. This transitory state should not be visible to lower privilege levels (i.e. supervisor domains). There are also security aspects to be considered during (re)configuration, e.g., clearing memory used by the current SD before assigning it to another SD. Refer to the RISC-V CoVE [2] ABI and threat model as a reference.

A hart/device may perform accesses to memory exclusively accessible to its supervisor domain, or to memory shared globally with one or more supervisor domains. Memory sharing between supervisor domains is achieved by simply making the physical memory region accessible to the supervisor domains via the MPT structure associated with the hart or device. Access to physical addresses initiated from a hart or a device assigned a supervisor domain identifier may be denied by virtue of the permissions in the MPT lookup - such disallowed accesses from a hart cause a trap to the RDSM to report a fault. In the case of a device access disallowed by the MPT, the IO sub-system may log an error for the RDSM which may delegate it to a supervisor domain.

The intra-domain isolation of memory between two harts/devices belonging to the same supervisor

domain, but different tenant workloads, may be achieved via the use of MMU, PMP/Smepmp, SPMP, IOMMU and IOPMP depending on the type of platform and the type of access. To successfully achieve this isolation, the page table structures for a domain's workloads must be managed by the Supervisor Domain Security Manager (SDSM) and the paging structures must be located in memory exclusively-accessible only to the supervisor domain. Additional security properties may be enforced based on type (data fetch, instruction fetch, etc.) and locality (hart supervisor domain identifier) of memory accesses as required for the security policy specific to usages. An example policy may be to require certain accesses to target only exclusively-owned domain memory. The MPT checker may utilize the supervisor domain identifier or additional metadata for the access to enforce such policies. The description of different types of supervisor domain policies possible is outside the scope of this document.

Additional protection/isolation for memory associated with a supervisor domain is orthogonal (and usage-specific). Such additional protection for memory may be derived by the use of cryptography and/or access-control mechanisms. The mechanisms chosen for these additional protection methods are independent of `Smmpt` and may be platform-specific. The TCB of a particular supervisor domain (and devices that are bound to it) may be independently evaluated via attestation of the HW and SW TCB by a relying party using standard Public-Key Infrastructure-based mechanisms.

2. Summary of extensions for Supervisor Domain Access Protection

The following normative architecture extensions are defined.

- `Smsdid` (Chapter 3) - An interface to program the active supervisor domain under which a hart is operating. This is a dynamic control state on the hart that is held in a machine-mode CSR and modifiable by the RDSM via CSR r/w instructions - herewith called the `supervisor domain identifier` assigned to the hart. The SDID is a local identifier for the hart and may be used to tag hart-local resources to access-control data associated with the supervisor domain. The supervisor domain identifier is independent from the hart privilege levels and is held in a machine-mode CSR. This extension may be used independently or may be combined with other extensions in this specification.
- `Smmpt` (Chapter 4) - An extension to set the access permissions for a memory region or page associated with a supervisor domain. This extension allows for dynamic changes of access permission. Such dynamic changes may require flushing of appropriate state cached in harts. The access properties are programmed via a Machine-level Memory Protection Tables (MPT) structure. The physical page number (PPN) of the root table of the MPT is programmed into a machine-mode CSR. When `Smmpt` is implemented, MPT and e(PMP) are always active. Although there is no option to disable MPT, it can be effectively disabled if granular memory access control is not required by configuring MPT mode to be `Bare`.
- `IO-MPT` (Chapter 5) - A non-ISA extension that enables programming of an IO interconnect to associate an IOMMU and devices in scope of that IOMMU with a SD. The assignment of IOMMUs to supervisor domains is also expected to be under the purview of the RDSM. IO-MPT extension specifies the memory access control mechanisms for memory accesses performed by the IOMMU as well as by the devices associated with that SD. Note that isolation of data within a device is out of scope of this specification.
- `Smsdia` (Chapter 6) - This extension enables assignment of IMSIC interrupt file(s) or an APLIC domain to a supervisor domain. The extension also provides CSRs to allow machine-mode software to retain control on notification of interrupts when Supervisor domains are enabled.
- `Smsdedbga` (Chapter 7) - This extension provides the controls to indicate if external debug is allowed for a supervisor domain. Whether external debug is authorized or not is expected to be done via a root of trust (RoT) and is outside the scope of this specification.
- `Smsdetrca` (Chapter 7) - This extension provides the controls to indicate if external trace is allowed for a supervisor domain. Whether external trace is authorized or not is expected to be done via a root of trust (RoT) and is outside the scope of this specification.
- `Smsqosid` and CBQRI for Supervisor Domains ([Smsdqos]) - This extension provides an interface for the RDSM to enforce that resource accesses from a supervisor domain or the RDSM must not be observable by entities that are not within their TCB using the resource usage monitors. Similarly, the resource allocations for a supervisor domain or the RDSM must not be influenced by entities outside their TCB.

3. Smsdid: Supervisor Domain Identifier and Protection Register

Smsdid defines an interface to program the configuration for supervisor-domain-related extensions. The interface consists of M-mode CSRs msdcfg and mmpt. When Smsdid is implemented, both CSRs must be implemented; msdcfg may be read-only zero if none of the optional supervisor domain extensions are implemented.

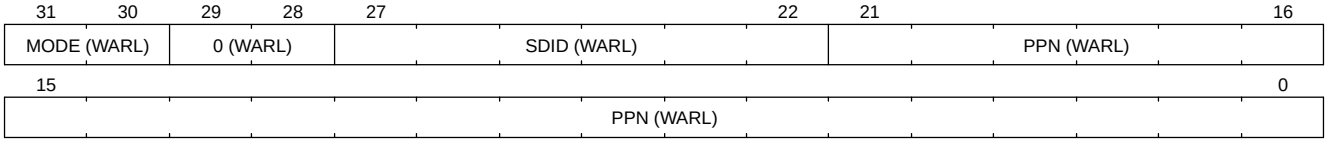
The SDID programmed via the mmpt CSR is a local identifier for the hart and may be used to tag hart-local resources to access-control data associated with the supervisor domain. The SDID is unique only to a hart, hence the SDID must not be used to tag downstream transactions that are globally visible.



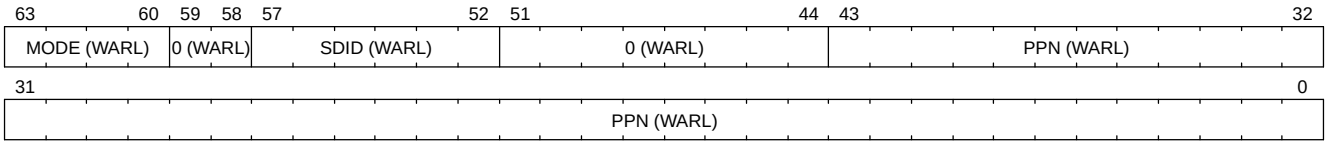
For most typical usages, the SDID will remain stable over the life of the supervisor domain. However, the SDID can be changed at the cost of flushing any prior cached state that was associated with the SDID.

3.1. Machine-level Memory Protection Tables (mmpt)

The mmpt register is an MXLEN wide read/write machine-mode register, formatted as shown in Register 1 for MXLEN=32 and Register 2 for MXLEN=64, which controls page-based physical address space protection. This register holds the supervisor domain identifier (SDID); the physical page number (PPN) of the root page of the memory protection tables (MPT), which facilitates page-based address space protection; and the MODE field, which selects the memory protection scheme for physical addresses. The CSR number for mmpt is 0x382.



Register 1: MMPT register (mmpt) when MXLEN=32.



Register 2: MMPT register (mmpt) when MXLEN=64.

The number of implemented SDID bits is UNSPECIFIED and may be zero. The number of implemented SDID bits, termed SDIDLEN, may be determined by writing one to every bit position in the SDID field, then reading back the value in mmpt to see which bit positions in the SDID field hold a one. The least-significant bits of SDID are implemented first: that is, if SDIDLEN > 0, SDID[SDIDLEN-1:0] is writable. The maximal value of SDIDLEN, termed SDIDMAX, is 6.

Table 1 shows the encodings of the MODE field when MXLEN=32 and Table 2 shows the encodings of the MODE field when MXLEN=64.

Table 1. Encoding of mmpt MODE field for MXLEN=32.

Value	Name	Description
0	Bare	No page-based memory protection beyond the physical memory protection scheme described in Section 3.7 of the RISC-V privileged architecture specification [3]. The remaining fields in mmpt (SDID, PPN) must be set to zeros.
1	Smmpt34	Page-based memory protection for up to 34-bit physical address spaces.
2	-	Reserved for future standard use.
3	-	Designated for custom use.

Table 2. Encoding of mmpt MODE field for MXLEN=64.

Value	Name	Description
0	Bare	No page-based memory protection beyond the physical memory protection scheme described in Section 3.7 of the RISC-V privileged architecture specification [3]. The remaining fields in mmpt (SDID, PPN) must be set to zeros.
1	Smmpt43	Page-based memory protection for up to 43-bit physical address spaces.
2	Smmpt52	Page-based memory protection for up to 52-bit physical address spaces.
3	Smmpt64	Page-based memory protection for up to 64-bit physical address spaces.
4-13	-	Reserved for future standard use.
14-15	-	Designated for custom use.

Implementations are not required to support all defined MODE settings. A write to mmpt with an unsupported MODE value is not ignored. Instead, the fields of mmpt are WARL in the normal way, when so indicated in Register 1 and Register 2.

mmpt.PPN refers to the root of the memory protection table when mmpt.MODE is not Bare.

When mmpt.MODE is Smmpt64, the root page of the memory protection table must be aligned to a 32 KiB boundary. In this mode, bits 2:0 of mmpt.PPN always read as zero.

The mmpt register is considered active for the purposes of access type permission lookup algorithm unless the effective privilege mode is M.

Note that writing mmpt does not imply any ordering constraint between updates of virtual memory system page-tables or memory protection tables and subsequent address translations or memory protection.

If an MPT structure for an address-space has been modified, or if a SDID is reused, it may be necessary to execute a MFENCE.PA instruction after, or in some cases before, writing mmpt.

3.2. Machine Memory Protection Fence Instruction

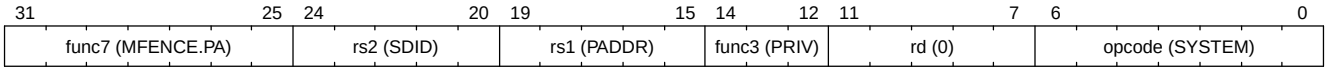


Figure 3: MFENCE.PA instruction

The `MFENCE.PA` fence instruction is used to synchronize updates to memory protection tables with current execution of supervisor domains. `MFENCE.PA` is only valid in M-mode. If operand `rs1 != x0`, it specifies a single physical address, and if `rs2 != x0`, it specifies a single `SDID`.

The behavior of `MFENCE.PA` depends on `rs1` and `rs2` as follows:

- If `rs1=x0` and `rs2=x0`, the fence orders all reads and writes to any level of the MPTs for all supervisor domain address spaces.
- If `rs1=x0` and `rs2!=x0`, the fence orders all reads and writes to any level of the MPT for the supervisor domain address space identified by the `SDID` in `rs2`.
- If `rs1!=x0` and `rs2=x0`, the fence orders all reads and writes made to the leaf MPT entries that correspond to the physical address in `rs1`, for all supervisor domain address spaces.
- If `rs1!=x0` and `rs2!=x0`, the fence orders all reads and writes made to the leaf MPT PTEs that correspond to the physical address in `rs1`, for the supervisor domain address space identified by the `SDID` in `rs2`.

Executing a `MFENCE.PA` guarantees that any previous stores already visible to the current hart are ordered before all implicit reads by that hart to memory protection tables.

When `SDID` is specified in `rs2`, bits `XLEN-1:SDIDMAX` held in `rs2` are reserved for future standard use. Until their use is specified, they should be zeroed by software and ignored by implementations. Also, if `SDIDLEN < SDIDMAX`, the implementation shall ignore bits `SDIDMAX-1:SDIDLEN` of the value held in `rs2`.



A simpler implementation of `MFENCE.PA` may ignore the physical address in `rs1`, and/or the `SDID` value in `rs2`, and always perform a global fence for all SDs.

3.3. M-mode Supervisor Domain Fine-Grain Invalidation Instruction

When `Svinval` and `Smsdid` are both implemented, the `MINVAL.PA` instruction must be implemented. `MINVAL.SPA` supports batching of MPT invalidations, analogous to `SINVAL.VMA`.

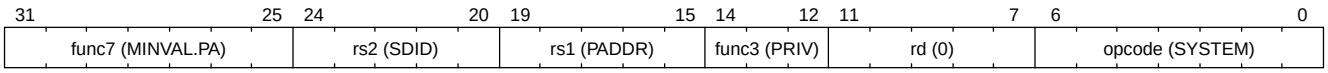


Figure 4: MINVAL.PA instruction

`MINVAL.PA` is only ordered against `SFENCE.W.INVAL` and `SFENCE.INVAL.IR` instructions.

The `SFENCE.W.INVAL` instruction guarantees that any previous stores already visible to the current RISC-V hart are ordered before subsequent `MINVAL.PA` instructions executed by the same hart.

The `SFENCE.INVAL.IR` instruction guarantees that any previous `MINVAL.PA` instructions executed by the current hart are ordered before subsequent implicit references by that hart to memory-

protection data structures.

When executed in order (but not necessarily consecutively) by a single hart, the sequence SFENCE.W.INVAL, MINVAL.PA and SFENCE.INVAL.IR has the same effect as a hypothetical MFENCE.PA in which:

- the values of rs1 and rs2 for the MFENCE.PA are the same as those used in the MINVAL.PA,
- reads and writes prior to the SFENCE.W.INVAL are considered to be those prior to the MINVAL.PA, and
- reads and writes following the SFENCE.INVAL.IR are considered to be those subsequent to the MFENCE.PA

MINVAL.PA is only valid in M-mode.

3.4. Extension to Behavior of Memory-Management Fences

The behavior of the SFENCE.VMA, HFENCE.GVMA, and HFENCE.VVMA instructions is affected when the Smsdid extension is implemented. When the Svinval extension is also implemented, the behavior of the SINVAL.VMA, HINVAL.GVMA, and HINVAL.VVMA instructions is also affected.

The ASID and VMID arguments of these instructions correspond to the ASID's and `VMID`s that are active within the current supervisor domain, which is identified by the `SDID` field of the CSR mmpt. The SDID augments the ASID and VMID values to form the effective identifiers used by these instructions.

An effective S/HS-level ASID can be considered to be the combination of the SDID with the S/HS-level ASID. Likewise, an effective VS-level ASID can be considered to be the combination of the SDID with the VS-level ASID.

An effective VMID can be considered to be the combination of the SDID with the VMID.

3.5. Machine supervisor domain configuration (msdcfg)

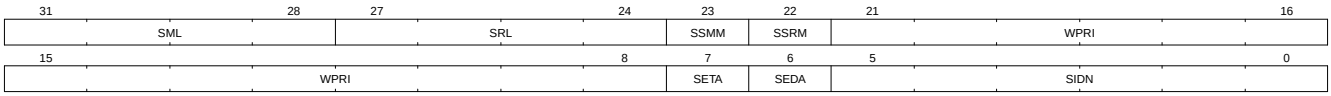
The msdcfg is a MXLEN wide machine-mode read/write register, formatted as shown in Register 3. This CSR is used to hold configurations for a supervisor domain. The CSR number for msdcfg is 0x74E.

The following extensions specify the fields of the msdcfg register:

1. Smsdia specifies the SIDN field to identify the *supervisor interrupt domain* associated with the hart.
2. Smsdedbga specifies the SEDA bit to allow external-debug for supervisor domains.
3. Smsdetrca specifies the SETA bit to allow external-trace for supervisor domains.
4. Smsdqosid specifies the SRL, SML, SSRM, and SSMM fields to determine supervisor domain QoS IDs.

Details of Smsdia, Smsdedbga, Smsdetrca, and Smsdqosid are described in their respective sections in

this specification. If an extension is not implemented, the corresponding configuration bits in `msdcfg` are read-only zero. Hence if no optional extensions are implemented, this CSR becomes read-only zero.



Register 3: `msdcfg` register

4. Machine-mode Memory Protection Table (MPT)

Memory Protection table is a radix-tree table used to provide read, write, and execute permissions for implicit and explicit memory accesses by effective privilege modes less than M.

The following MPT extensions are defined:

- Smmpt34 - Page-based 34-bit memory protection system (RV32 only)
- Smmpt43 - Page-based 43-bit memory protection system (RV64 only)
- Smmpt52 - Page-based 52-bit memory protection system (RV64 only)
- Smmpt64 - Page-based 64-bit memory protection system (RV64 only)

These MPT extensions depend on `Smsdid`.

4.1. Smmpt34: Page-Based Memory Protection System

The RV32 Memory Protection Table provides memory access type permissions for pages in a 34-bit supervisor physical address (SPA) space. The SPA is partitioned into a page number (`pn`) and a range offset. When the Smmpt34 mode is selected in the `MODE` field of `mmpt`, the access type permissions for a supervisor physical address are determined using a two-level memory protection table.



Figure 5: RV32 Supervisor Physical Address

The root memory protection table has 2^9 memory-protection-table entries (MPTEs) and the memory protection tables linked to the root table have 2^{10} MPTEs. Each MPTE is a four byte value. Each memory protection table, including the root MPT, must always be aligned to a page boundary.

If the MPTE points to the next level of the memory protection table, it is a non-leaf MPTE, else it is a leaf MPTE. A leaf MPTE provides access type permissions for eight pages in the address range determined by that MPTE.

The format of a non-leaf MPTE entry is shown in Figure 6 and that of a leaf MPTE entry in Figure 7.

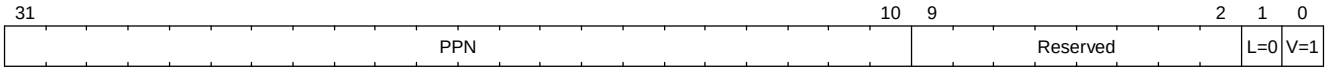


Figure 6: `Smmpt34` non-leaf `MPTE`

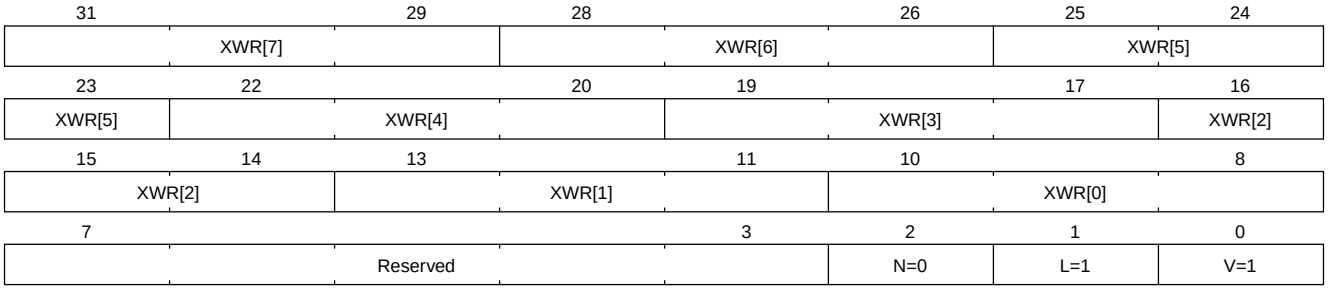


Figure 7: Smmpt34 non-NAPOT leaf MPTE

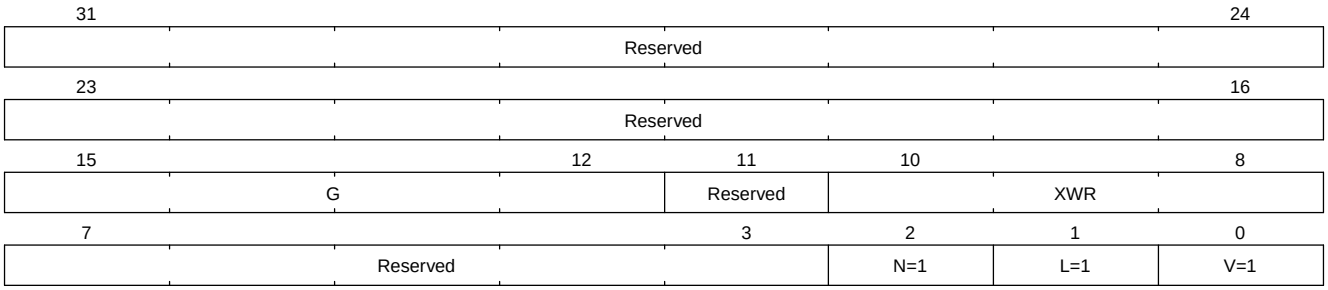


Figure 8: Smmpt34 NAPOT leaf MPTE

The V bit indicates if the MPTE is valid; if it is 0, all other bits in the MPTE are don't-cares and may be used freely by software. When the MPTE is valid, the bits constituting the RESERVED field are reserved for future standard use and, until its use is defined by some standard extension, must be zeroed by software for forward compatibility. If any of these bits are set, an access-fault exception is raised.

Explicit stores by a hart that update the V bit of leaf and/or non-leaf MPTEs from 0 to 1 and are visible to a hart will eventually become visible within a bounded timeframe to subsequent implicit accesses by that hart to such MPTEs.



This behavior relieves M-mode software from executing certain memory-management instructions, such as MFENCE.PA or MINVAL.PA, which would normally be used to synchronize the hart's MPT caches when a memory-resident MPTE is changed from Invalid to Valid. Synchronizing the hart's MPT caches with other forms of updates to a memory-resident MPTE, including when a MPTE is changed from Valid to Invalid, requires the use of suitable memory-management instructions. This behavior guarantees that a change to a MPTE from Invalid to Valid is made visible within a bounded time, thereby making the execution of these memory-management instructions redundant. The performance benefit of eliding these instructions outweighs the cost of an occasional gratuitous access fault that may occur.

Depending on the microarchitecture, some possible ways to facilitate include: not storing Invalid PTEs in the MPT page-walk caches, automatically evicting Invalid MPTEs using a bounded timer, or making MPT page-walk caches coherent with store instructions that modify MPTEs.

When the L bit is set to 0, the entry is a non-leaf MPTE, else it is a leaf MPTE.

When MPTE.L=1 and MPTE.N=1, the leaf MPTE represents a memory protection range that is part of a

larger contiguous NAPOT memory protection range comprised of 2^(G+1) MPTEs. All leaf MPTEs at that level of this NAPOT memory protection range have the same value for L, N, XWR and V bits. The encodings 0-5 and 7-15 of G are reserved for Smmpt34. The format of a NAPOT leaf MPTE is shown in Figure 8.



The motivation of NAPOT MPTE is that access-type permissions for one or more entries representing the contiguous region may be cached as a single cache entry as if it were a single (large) page. This compaction can relieve access-type permission caches. The scheme allows an implementation to not take advantage of this property and simply cache the access-type permissions for pages separately.

The encoding of G=6 supports caching a single access-type permission entry representing a 4 MiB address range. Depending on need, the NAPOT scheme may be extended to other page sizes in the future. The encoding is designed to accommodate other NAPOT sizes should that need arise.

A non-leaf MPTE holds the PPN of the next level of the memory protection table.

A leaf MPTE holds eight tuples of access type permission bits - one for each page in the range determined by that MPTE. Each tuple consists of 3 bits XWR, that provide the Execute (X), Write (W), and Read (R) permissions. The most significant 3 bits of the range offset identify a page in that range and are used to identify the access type permission bits for an access to that page.

Table 3. Encoding of MPTE X/W/R permission tuple

X	W	R	Meaning
0	0	0	No access.
0	0	1	Read-only page.
0	1	0	*Reserved for future use.*
0	1	1	Read-write page.
1	0	0	Execute-only page.
1	0	1	Read-execute page.
1	1	0	*Reserved for future use.*
1	1	1	Read-write-execute page.

Access type permissions — readable, writable, or executable — are checked by MPT the same as for VS-stage and G-stage translation. For a memory access made to support VS-stage or G-stage address translation (such as a read/write to a VS-stage page table or a G-stage page table), permissions are checked as though for an implicit load or store, not for the original access type. However, any exception is always reported for the original access type (instruction, load, or store/AMO).

Any level of MPTE may be a leaf MPTE, so in addition to providing access type permissions to 4 KiB pages, Smmpt34 supports 4 MiB pages.

4.1.1. MPT access type permissions lookup process

Access type permissions for a physical address PA are determined as follows:

1. Let *a* be `mmpt.ppn` x PAGESIZE, and let *i* = LEVELS-1 (For Smmpt34, PAGESIZE is 2^{12} and LEVELS=2). The `mmpt` register must be active, i.e., the effective privilege mode must not be M-mode.
2. Let *mpte* be the value of the `MPTE` at address *a* + *pa.pn[i]* x MPTESIZE (For Smmpt34, MPTESIZE=4). If accessing *mpte* violates a PMA or PMP check, raise an access-fault exception corresponding to the original access type.
3. If *mpte.V*=0, or if any bits or encodings that are reserved for future standard use are set within *mpte*, or if *mpte.L* = 0 and *mpte.N* = 1, stop and raise an access-fault exception corresponding to the original access type.
4. Otherwise, the *mpte* is valid. If *mpte.L=1* go to step 5; Otherwise, this MPTE is a pointer to the next level of the memory protection table. Let *i*=*i*-1. If *i*<0, stop and raise an access-fault exception corresponding to the original access type. Otherwise, let *a* = *mpte.PPN* x PAGESIZE and go to step 2.
5. If *mpte.N*=0, a non-NAPOT leaf *mpte* has been found. If *i* > 0, let *pi* be the value of the NUMPGINRANGE most significant bits of *pa.pn[i-1]*. Otherwise, for *i* = 0, let *pi* be the value of the NUMPGINRANGE most significant bits of the range offset specified in the *pa*. (For Smmpt34, NUMPGINRANGE is 3). Let *XWR=mpte.XWR[pi]*.
6. If *mpte.N*=1, a NAPOT leaf *mpte* has been found. Let *XWR=mpte.XWR*. If the value of *G* is reserved for the current MPT mode, raise an access-fault exception corresponding to the original access type.
7. Determine if the requested memory access is allowed by the *XWR* bits If not, stop and raise an access-fault exception corresponding to the original access type, else the requested memory access is allowed.
    - The MXR mechanisms or the use of HLVX instruction cannot be used to affect or override the interpretation of permissions encoded in memory-protection-table entries.
    - When Zicfiss extension is also implemented, the memory accessed by the shadow stack instructions require read-write permission in MPT entries.

Implicit accesses to MPT by this process are checked by PMP/Smepmp as implicit M-mode accesses. The endianness of such implicit accesses is controlled by `mstatus.MBE`.

4.2. Smmpt43: Page-Based 43-bit Memory Protection System

This section describes a page-based memory protection system for RV64, which supports a 43-bit supervisor physical address space. The design of Smmpt43 follows the overall scheme of Smmpt34, and this section details only the differences between the schemes.



Multiple memory protection systems are specified for RV64 to relieve the tension between providing a large physical address space and minimizing the memory

protection cost. For many systems, 43-bits of physical address space is ample, and so Smmpt43 suffices. Smmpt52 increases the physical address space to 52 bits, but increases the physical memory capacity dedicated to memory protection tables and the latency of memory protection table traversals. Smmpt64 increases the physical address space to 64 bits.

Smmpt43 supports a 43-bit supervisor physical address space, divided into pages. The Smmpt43 address is partitioned as shown in Figure 9. The supervisor physical address must have bits 43 and higher all equal to 0, or else an access-fault exception corresponding to the original access type will occur. The Smmpt43 memory protection table has three levels.

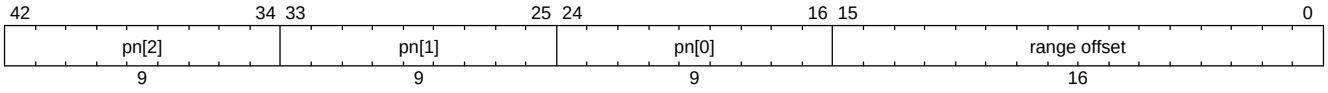


Figure 9: Smmpt43 Supervisor Physical Address

The Smmpt43 memory protection tables contain 2^9 memory protection table entries (MPTEs). Each Smmpt43 MPTE is an eight byte value. A page table is exactly the size of a page and must always be aligned to a page boundary. The physical page number of the root memory protection table is stored in the `mmpt` register's PPN field.

The format of a non-leaf MPTE entry is shown in Figure 10 and that of a leaf MPTE entry in Figure 11. The format of a NAPOT leaf MPTE is shown in Figure 12.

A leaf MPTE provides access type permissions for sixteen pages in the address range determined by that MPTE.



Figure 10: Smmpt43 non-leaf `MPTE` entry

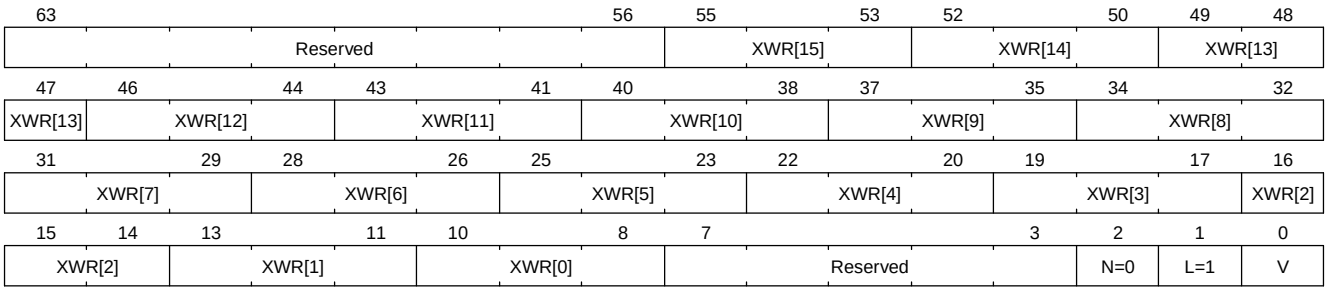


Figure 11: Smmpt43 non-NAPOT leaf `MPTE` entry

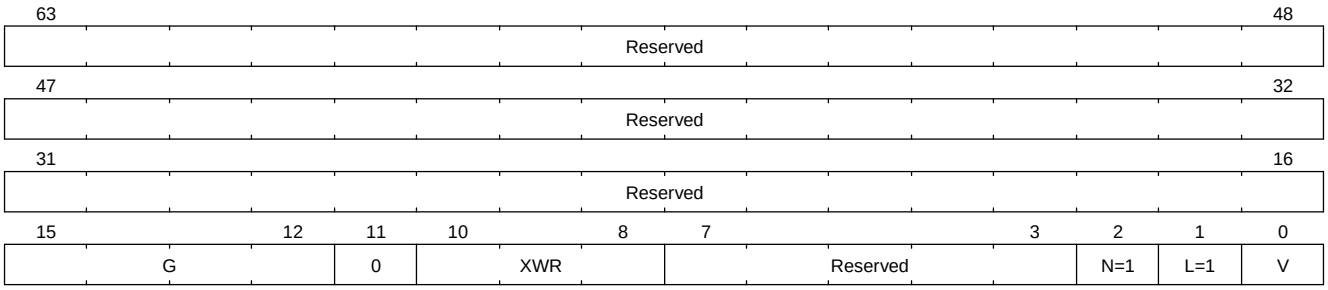


Figure 12: Smmpt43 NAPOT leaf `MPTE` entry

Any level of MPTE may be a leaf MPTE, so in addition to providing access type permissions to 4 KiB pages, Smmpt43 supports 2 MiB, and 1 GiB pages.

The algorithm to determine access type permissions for a page is same as in Section 4.1.1, except LEVELS equals 3, MPTESIZE equals 8, and NUMPGINRANGE equals 4.

The encodings 0-3 and 5-15 of `G` are reserved for Smmpt43.



The encoding of G=4 supports caching a single access-type permission entry representing a 2 MiB or a 1 GiB address range. These contiguous address range sizes represent large/huge page sizes used commonly by memory allocators.

Depending on need, the NAPOT scheme may be extended to other page sizes in the future.

4.3. Smmpt52: Page-Based 52-bit Memory Protection System

This section describes a page-based memory protection system for RV64 that supports 52-bit supervisor physical address spaces. It closely follows the design of Smmpt43, simply adding an additional level of page table, and so this section only details the differences between the two schemes.

Smmpt52 supports a 52-bit supervisor physical address space, divided into pages. The Smmpt52 address is partitioned as shown in Figure 13. The supervisor physical address must have bits 52 and higher all equal to 0, or else an access-fault exception corresponding to the original access type will occur. The Smmpt52 memory protection table has four levels.

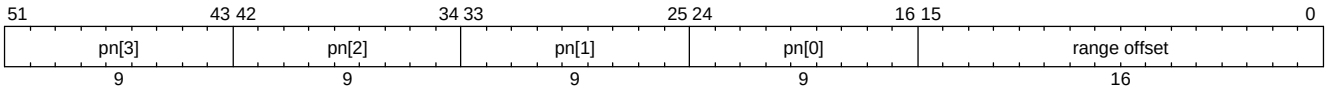


Figure 13: Smmpt52 Supervisor Physical Address

The Smmpt52 memory protection tables contain 2^9 memory protection table entries (MPTEs). Each MPTE is an eight byte value. A page table is exactly the size of a page and must always be aligned to a page boundary. The physical page number of the root memory protection table is stored in the `mmpt` register's PPN field.

The MPTE formats for Smmpt52 are identical to that of Smmpt43.

Any level of MPTE may be a leaf MPTE, so in addition to providing access type permissions to 4 KiB pages, Smmpt52 supports 2 MiB, 1 GiB, and 512 GiB pages.

The algorithm to determine access type permissions for a page is same as in Section 4.1.1, except LEVELS equals 4, MPTESIZE equals 8, and NUMPGINRANGE equals 4.

The encodings 0-3 and 5-15 of `G` are reserved for Smmpt52.

4.4. Smmpt64: Page-Based 64-bit Memory Protection System

This section describes a page-based memory protection system for RV64 that supports 64-bit supervisor physical address spaces. It closely follows the design of Smmpt52, simply adding an additional level of page table, and so this section only details the differences between the two schemes.

Smmpt64 supports a 64-bit supervisor physical address space, divided into pages. The Smmpt64 address is partitioned as shown in Figure 14. The Smmpt64 memory protection table has five levels.

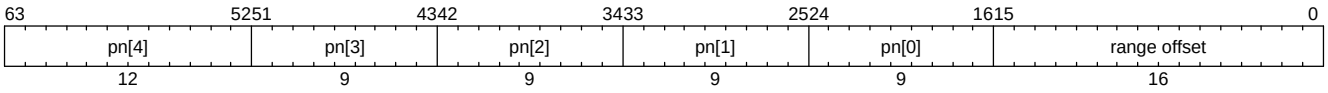


Figure 14: Smmpt64 Supervisor Physical Address

The Smmpt64 root memory protection table contain 2^{12} memory protection table entries (MPTEs). Each MPTE is an eight bytes value. The root memory protection table is 32 KiB instead of the usual 4 KiB and must be aligned to a 32 KiB boundary. The physical page number of the root memory protection table is stored in the `mmpt` register's PPN field.

The non-root Smmpt64 memory protection tables contain 2^9 MPTEs, eight bytes each. The non-root memory protection table is exactly the size of a page and must always be aligned to a page boundary.

The MPTE formats for Smmpt64 are identical to that of Smmpt52.

Any level of MPTE may be a leaf MPTE, so in addition to providing access type permissions to 4 KiB pages, Smmpt64 supports 2 MiB, 1 GiB, 512 GiB, and 256 TiB pages.

The algorithm to determine access type permissions for a page is same as in Section 4.1.1, except LEVELS equals 5, MPTESIZE equals 8, and NUMPGINRANGE equals 4.

The encodings 0-3 and 5-15 of `G` are reserved for Smmpt64.

5. I/O MPT Checker

Supervisor domains may be granted control over DMA-capable devices. When such direct device association is supported, the system might also incorporate multiple instances of IOMMU. Each IOMMU instance can be tied directly to a supervisor domain, allowing that domain to manage address translation and protection for DMA that originates from devices under its control.

To uphold isolation properties, the DMA from the devices and the IOMMU linked with a supervisor domain must adhere strictly to the access protections encoded in the MPT of the respective supervisor domain. Additionally, using the MPT, the RDSM enforces that the IOMMU memory-mapped programming regions are access-restricted to the supervisor domain the IOMMU is assigned to.

At any given time, a solitary supervisor domain is scheduled for execution on a RISC-V hart by the root domain security manager (RDSM). As part of this scheduling, the RDSM programs a pointer to the MPT into a CSR within the hart. Unlike the RISC-V harts, DMA-capable devices connected to a supervisor domain remain continuously active. Such devices might initiate DMA even if the associated domain is not currently active on any RISC-V harts. As a result, the MPT of all supervisor domains must be constantly active for DMA protection. Furthermore, the input/output (I/O) subsystem must possess the capability to select the appropriate MPT for enforcement based on the identity of the device initiating the DMA.

Given this setup, the I/O subsystem is required to offer the following functions:

- Supervisor Domain Classifier (SDCL): This function interprets the attributes of a DMA request and determines the appropriate MPT for that request.
- MPT Check: This function ensures that stipulated access controls by `Smmpt` are applied to the memory regions accessed by the DMA. It uses the MPT identified by the SDCL.

Collectively, these two functionalities form a logical block in the I/O subsystem, referred to as the I/O MPT Checker.

5.1. Placement of I/O MPT Checker

The I/O Bridge serves as an intermediary, situated between the I/O devices and the system interconnect, with the primary role of processing DMA transactions. These I/O devices can initiate DMA transactions utilizing I/O Virtual Addresses (IOVA). Notably, an IOVA could be in the form of a Virtual Address (VA), Guest Virtual Address (GVA), or Guest Physical Address (GPA). The configuration and interfacing of the I/O MPT Checker with respect to the I/O Bridge is graphically represented in Figure 15.

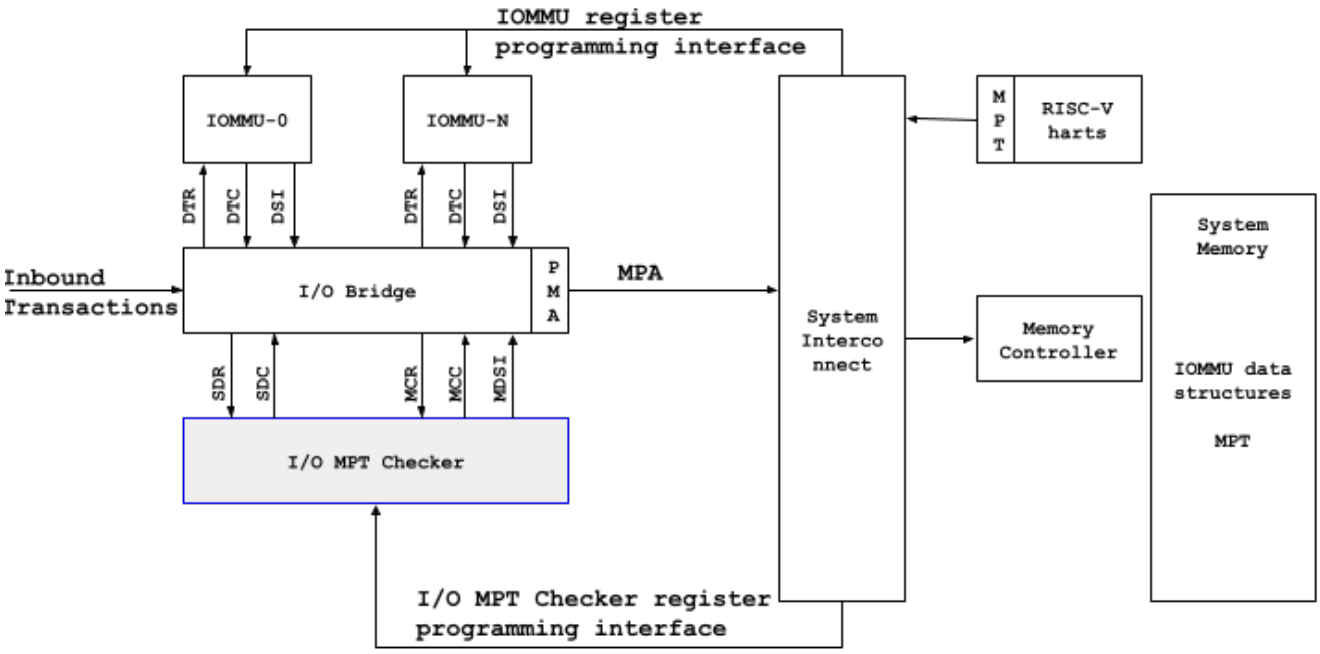


Figure 15: I/O MPT checker placement

The I/O Bridge invokes the I/O MPT Checker using the SDID request interface (SDR) and provides it the identifiers associated with the incoming transaction. The SDCL classifies the request using the identifiers and provides the SDID and the IOMMU ID on the SDID completion interface (SDC).

The I/O Bridge uses the IOMMU ID to determine the IOMMU governing this request. The I/O Bridge uses the device translation request (DTR) interface to the selected IOMMU to request address translation. The selected IOMMU provides the response to the address translation request on the device translation completion (DTC) interface. As part of the address translation process, the IOMMU may need to access its in-memory data structures over its data structure interface (DSI).

The I/O Bridge invokes the I/O MPT Checker over the MPT check request (MCR) interface and provides it the SDID and supervisor physical address of the access. The I/O MPT Checker uses the SDID to determine the MPT associated with the supervisor domain and checks if the supervisor physical address may be accessed by the device or IOMMU associated with that supervisor domain. The result of the check is provided on the MPT check completion (MCC) interface to the I/O Bridge. As part of the MPT check, the I/O MPT Checker may need to perform implicit accesses to the MPT using the I/O MPT Checker data structure interface (MDSI). To perform the checks the I/O MPT Checker uses the same MPT table format as used by the CPU's MMU. Using the same MPT table formats as the CPU's MMU allows the same table to be used simultaneously by the CPU MMU and I/O MPT Checker. The MPT permission lookup process used by I/O MPT Checker is identical to that specified by `Smmpt` extension in section "MPT access permissions lookup process".

The RISC-V memory model requires memory access from a hart to be single-copy atomic. When

RV32 is implemented the size of a single-copy atomic memory access is up to 32-bits. When RV64 is implemented the size of a single-copy atomic memory access is up to 64-bits. The size of a single-copy atomic memory access implemented by I/O MPT Checker is UNSPECIFIED but is required to be at least 32-bits if all of the harts in the system implement RV32 and is required to be at least 64-bits if any of the harts in the system implement RV64.

The MPT entries have a `V` bit that when set to 1 indicates that the entry is valid.

Software is allowed to make updates to a MPT entry that has the `V` bit set to 1. However, some rules as outlined must be followed.

- Software must use a single atomic store that updates all or parts of an MPT PTE for the update to be atomically observed by the I/O MPT Checker.
- If the update to a field of the MPT entry will make the field inconsistent with another field of the entry then software must first set the `V` bit to 0 and use the `MPTINVAL` operation outlined in Section 5.6 to invalidate previous copies of the entry that may be in the I/O MPT Checker caches to synchronize the updates to the entry with the operation of I/O MPT Checker.
- I/O MPT Checker is not required to immediately observe the software updates to an MPT entry. Software must use the `MPTINVAL` operation outlined in Section 5.6 to invalidate any previous copies of that entry that may be in the I/O MPT Checker caches to synchronize the updates to the entry with the operation of I/O MPT Checker.



If an MPT entry is changed, I/O MPT Checker may use the old value of the entry or the new value of the entry and the choice is unpredictable until software uses the `MPTINVAL` operation to synchronize updates to the entry with the operation of the I/O MPT Checker. These are the only behaviors expected.

The RISC-V memory model specifies byte-invariance for the entire address space. When mixed-endian mode of operation is supported, the I/O MPT Checker must implement byte-invariant addressing such that a byte access to a given address accesses the same memory location in both little-endian and big-endian mode of operation.

The I/O MPT Checker is not allowed to cache an MPT entry whose `V` (valid) bit is clear.

The I/O MPT Checker provides a memory-mapped register programming interface.

As part of its operations, I/O MPT Checker may need to read data from the MPT. The provider (a memory controller or a cache) of the data may detect that the data requested has an uncorrectable error and signal that the data is corrupted and defer the error to I/O MPT Checker. Such technique to defer the handling of the corrupted data to the consumer of the data is also commonly known as data poisoning. The effects of such errors may be contained to the transaction that caused the corrupted data to be accessed. In the cases where the error affects the transaction being processed but otherwise allows the I/O MPT Checker to continue providing service, the I/O MPT Checker may request the I/O Bridge to abort the transaction. The I/O MPT Checker may support the RISC-V RAS error record register interface (RERI) that specifies methods for enabling error detection, logging the detected errors, and configuring means to report the error to an error handler. When such a RAS architecture is supported, errors such as attempted consumption of poisoned data may be reported using the methods provided by the RAS architecture.

A supervisor domain, identified by an SDID, may be associated with one or more IOMMU instances that are each identified by an IOMMU ID. A given IOMMU ID however can only be associated with one supervisor domain.

The IOMMU is itself a DMA-capable device, and performs DMA using its own device ID. An explicit rule must associate the IOMMU's device ID with an `SDID` and MPT, unless that device ID is already covered by another rule that maps device IDs to a supervisor domain. The IOMMU ID configured in the rule matching the IOMMU's device ID is ignored when the rule is looked up for memory accesses originated by the IOMMU itself.



Usually an IOMMU is associated with a root port, and a root port connects to one or more devices that are governed by that IOMMU. An I/O MPT Checker may be associated with more than one root port. A supervisor domain may be associated with one or more devices from each of these root ports, necessitating the association of an IOMMU instance for each root port to be used by that supervisor domain. When a common I/O MPT Checker is used to check I/O originating from multiple root ports, the I/O MPT Checker may be configured with rules to classify a device connected to a root port as associated with a supervisor domain and with the IOMMU instance that governs that device.

To support MPT checks, the RDSM provides two types of configurations to the I/O MPT Checker:

- SDCL rules that classify one or more devices to an SDID and an IOMMU ID.
- Configurations that determine the MPT used to check memory requests originated by the IOMMUs and devices associated with that SDID, and configurations that determine the QoS IDs carried with such memory requests.

These configurations are programmed through the I/O MPT Checker register interface.

While the common use case involves the use of an IOMMU, the architecture supports implementations without an IOMMU. In such implementations, the SDCL rules classify devices to an SDID alone and no IOMMU ID is associated with the rules.

5.2. Handling I/O MPT Checker violations

A transaction shall be aborted if:

- The I/O MPT Checker is in `Off` mode, or is in `Bare` mode and the transaction is TEE-associated.
- The I/O MPT Checker is in `On` mode but is not configured with a rule to classify the transaction.
- The memory targeted by the transaction is disallowed by the MPT.
- An access to the MPT by I/O MPT Checker is disallowed by a physical memory attributes check or a physical memory protection check performed by the IO Bridge.
- An access to the MPT by the I/O MPT Checker returns corrupted (poisoned) data.

If the aborted transaction is an IOMMU-initiated memory access then the IO Bridge signals such access faults to the IOMMU itself. The details of such signaling is implementation defined. Please see the IOMMU specification for the handling of access faults on IOMMU initiated memory accesses.

If the aborted transaction is a write then the I/O Bridge may discard the write; the details of how the write is discarded are implementation defined. If the IO protocol requires a response for write transactions (e.g., AXI) then a response as defined by the I/O protocol may be generated by the I/O Bridge (e.g., SLVERR on BRESP - Write Response channel). For PCIe, for example, write transactions are posted and no response is returned when a write transaction is discarded.

If the aborted transaction is a read then the device expects a completion. The I/O Bridge may provide a completion to the device. The data, if returned, in such completion is implementation defined; usually it is a fixed value such as all 0 or all 1. A status code may be returned to the device in the completion to indicate this condition. For AXI, for example, the completion status is provided by SLVERR on RRESP (Read Data channel). For PCIe, for example, the completion status field may be set to "Unsupported Request" (UR) or "Completer Abort" (CA).

Transactions permitted by the I/O MPT Checker may still be disallowed by additional checks outside the I/O MPT. The nature of such checks and the handling of their violations are outside the scope of this specification.

5.3. I/O MPT Checker Register Interface

Each I/O MPT Checker register interface is memory-mapped starting at an 8-byte aligned physical address and includes the registers used to configure the the I/O MPT Checker.



Implementations may choose to implement a coarser alignment for the start address of the register interface. For example, some implementations may locate the register interface within a naturally aligned 4-KiB region (a page) of physical address space for each register interface. Coarser alignments may enable register decoding to be implemented without a hardware adder circuit.

The behavior for register accesses where the address is not aligned to the size of the access, or if the access spans multiple registers, or if the size of the access is not 4 bytes or 8 bytes, is `UNSPECIFIED`. An aligned 4 byte access to a I/O MPT Checker register must be single-copy atomic. Whether an 8 byte access to an I/O MPT Checker register is single-copy atomic is `UNSPECIFIED`, and such an access may appear, internally to the I/O MPT Checker implementation, as if two separate 4 byte accesses were performed.



The I/O MPT Checker registers are defined in such a way that software can perform two individual 4 byte accesses, or hardware can perform two independent 4 byte transactions resulting from an 8 byte access, to the high and low halves of the register as long as the register semantics, with regards to side-effects, are respected between the two software accesses, or two hardware transactions, respectively.

The I/O MPT Checker registers have little-endian byte order (even for systems where all harts are big-endian-only).



Big-endian-configured harts that make use of I/O MPT may implement the `REV8` byte-reversal instruction defined by the Zbb extension. If `REV8` is not implemented,

then endianness conversion may be implemented using a sequence of instructions.

Table 4. I/O MPT Checker register layout

Offset	Name	Size	Description	Optional?
0	`capabilities`	4	Capabilities	No
4	`status`	4	Status	No
8	`control`	4	Control	No
12	`command`	4	Command	No
16	`data1`	8	Input/Output data 1	No
24	`data2`	8	Input/Output data 2	No

The reset value is 0 for the following registers fields.

- `status` - `BUSY` and `CODE` fields

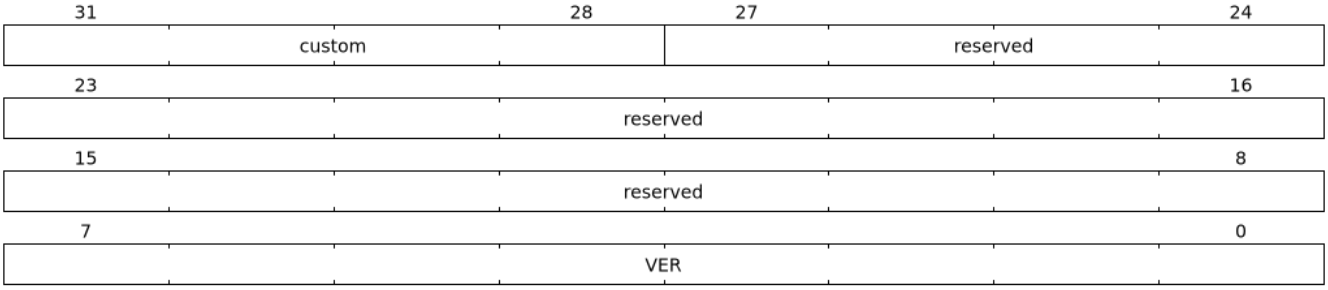
The reset value for `control.MODE` is recommended to be `Off`.

After a reset the MPT PTE caches in the I/O MPT Checker must have no valid entries.

The reset value is `UNSPECIFIED` for all other registers and/or fields.

5.4. Capabilities (`capabilities`)

The `capabilities` register is a read-only register that holds the I/O MPT Checker capabilities.



Register 4: Capabilities register fields

The `VER` field holds the version of the specification implemented by the I/O MPT Checker. The low nibble is used to hold the minor version of the specification and the upper nibble is used to hold the major version of the specification. For example, an implementation that supports version 1.0 of the specification reports 0x10.

5.5. Status register (`status`)

The `status` register provides the status of the operation requested using the `command` register.

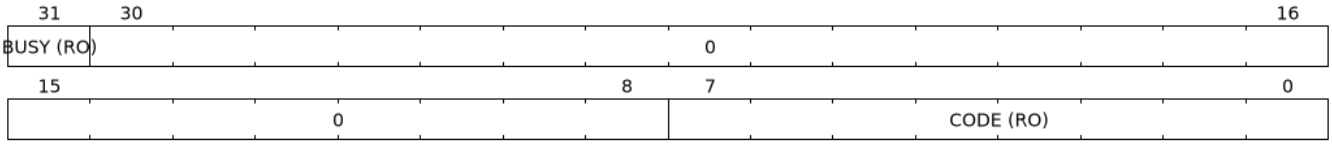


Figure 1. Status Register (`status`)

When the `command` register is written, the I/O MPT Checker may perform actions that do not complete synchronously with the write. A write to the `command` register sets the `status.BUSY` bit to 1, indicating that I/O MPT Checker is executing the requested operation. The behavior of writing the `command` or the `data*` registers while the `status.BUSY` bit is 1 is `UNSPECIFIED`. Some implementations may ignore the second write, while others may perform the operation specified by that write. Software must verify that `BUSY` is 0 before writing to the `control`, `command` or `data*` registers.



An implementation that always completes the requested operation synchronously with a write to the `command` or the `control` register may hardwire the `BUSY` bit to 0.

When the `status.BUSY` bit reads 0, the operation is complete, and the `status.CODE` field provides a status value (Table 5) for the requested operation.

Table 5. `status.CODE` field encodings

`STATUS`	**Description**
0	Reserved.
1	Operation completed successfully.
2	Invalid operation (`OP`) requested.
3	Operation requested for invalid `RULEID`.
4	Operation requested for invalid `SDID`.
5	Illegal or invalid operand encodings.
6-127	Reserved for future standard use.
128-255	Designated for custom use.

5.6. Control (`control`)

This register is used to control the operations of the I/O MPT Checker. An implementation may allow one or more fields in the register to be writable to support enabling or disabling the feature controlled by that field. All I/O MPT Checkers must support the `Off` and `On` values for the `MODE` field.

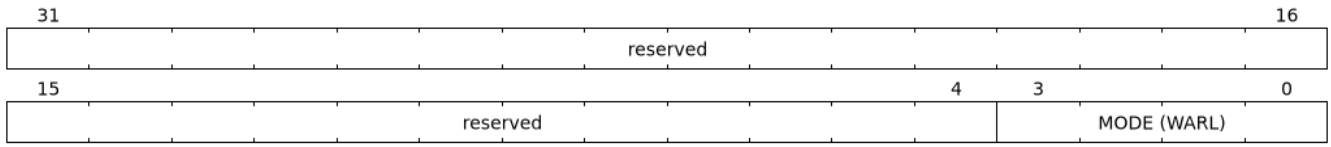


Figure 2. Control Register (`control`)

The I/O MPT Checker may be configured to be in the following modes:

Table 6. I/O MPT checker modes (`MODE`)

Name	Encoding	Description
Off	0	No inbound memory transactions are allowed.
Bare	1	No memory protection.
On	2	I/O MPT Checker is operational.
Reserved	3-13	Reserved for future standard use.
Custom	14-15	Designated for custom use.

When the `MODE` field value is changed to `Off` the I/O MPT Checker guarantees that in-flight transactions, observed at the time of the write to this field, from devices connected to the I/O MPT Checker will either be processed with the configurations applicable to the old value of the `MODE` field or are aborted as described in Section 5.2. It also ensures that all transactions and previous requests from devices that have already been processed by the I/O MPT Checker are committed to a global ordering point such that they can be observed by all RISC-V harts, devices, I/O MPT Checkers, and IOMMUs in the platform.

When an I/O MPT Checker is transitioned to `Off` state, the I/O MPT Checker may retain information configured into it. The I/O MPT Checker may retain information cached from the MPTs. Software must use suitable invalidation commands to invalidate the cached entries. Software must initialize all SDCL rules and SDID configurations prior to transitioning the I/O MPT Checker to `On` state.

When the `MODE` is not `Off`, software changes to fields besides `MODE` may cause UNSPECIFIED behavior. When the state is `On`, software must first change the `MODE` to `Off` without changing the values of any other field.

When the `MODE` is `Bare`, the I/O MPT Checker shall request the I/O Bridge to allow inbound transactions that are not TEE-associated and to abort those that are TEE-associated.

A write to `control` register may require the I/O MPT Checker to perform many operations that may not occur synchronously to the write. When a write is observed by `control`, the `status.BUSY` bit is set to 1. When the `BUSY` bit is 1, behavior of additional writes to `control` are UNSPECIFIED. Some implementations may ignore the second write and others may perform the actions determined by the second write. Software must verify that the `status.BUSY` bit is 0 before writing to the `control` register. If the `BUSY` bit reads 0 then the I/O MPT Checker has completed the operations associated with the previous write to the `control` register. The value held in `status.CODE` on completing the operations associated with a write to `control` register is UNSPECIFIED.

5.7. Command register (`command`)

The `command` register is used to initiate I/O MPT Checker operations. The specific operation is identified by the `OP` field within the `command` register. Operands required to complete the operation are supplied either in dedicated fields of the `command` register or in one or more of the associated `data*` registers. The `OP` field also defines the encoding of the `operand` fields in the `command` register and of the `data*` registers for that operation.

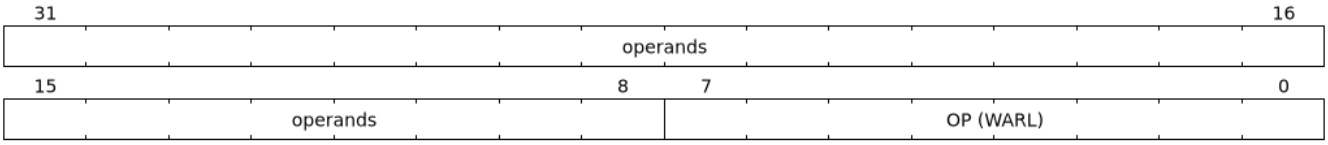


Figure 3. Command Register (command)

Table 7. I/O MPT Checker operations (OP)

Operation	Encoding	Description
—	0	Reserved for future standard use.
IOFENCE	1	Ensure that all previous read and write requests from devices already processed by the I/O MPT Checker are committed to a global ordering point such that they can be observed by all RISC-V harts, IOMMUs, and devices in the system.
SET_SDCL_ENTRY	2	Configure a SDCL rule in the I/O MPT Checker.
GET_SDCL_ENTRY	3	Read the configuration of an SDCL rule.
SET_SDCFG_ENTRY	4	Configure MPT and QoS ID parameters for an SD.
GET_SDCFG_ENTRY	5	Read the configuration associated with an SD.
MPTINVAL	6	Ensure that stores to an MPT are observed by the I/O MPT Checker before subsequent implicit reads by I/O MPT Checker to the corresponding MPT.
—	7–127	Reserved for future standard use.
—	128–255	Designated for custom use.

Before requesting an operation using the command register, software must program the fields of the data1 and data2 registers as required by that operation. Fields not used by an operation are ignored. If the command.OP encoding is invalid or not supported then the operation fails with "Invalid operation (OP) requested" (status.CODE=2). If the SDID or the RULEID operands are invalid then the operation fails with "Operation requested for invalid SDID " (status.CODE=3) or "Operation requested for invalid RULEID " (status.CODE=4) respectively. If other register fields used by the operation have an unsupported or invalid encoding then the operation fails with "Illegal or invalid operand encodings" (status.CODE=5). The order of checking the validity of SDID, RULEID, and other operands of when a valid operation is requested operation is UNSPECIFIED.

5.7.1. Configure a SDCL rule - SET_SDCL_ENTRY

The SET_SDCL_ENTRY operation configures an SDCL rule to classify DMA requests based on the identifiers associated with the requests, in order to determine the supervisor domain ID (SDID) and the IOMMU ID (IOMMU_ID). The rule to be configured is identified by RULEID. The RULEID, IOMMU_ID, and SDID fields must contain values in the range 0 to N–1, where N is the maximum number of the corresponding identifiers supported by the implementation.

This operation uses the operands in the command register and the data1 register. The contents of the data2 register are ignored by this operation.

For this operation, the fields of the command and data1 registers are interpreted as follows.

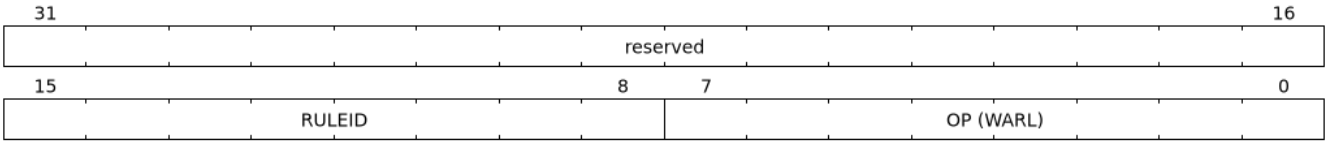


Figure 4. `command` register layout for `SET_SDCL_ENTRY` operation

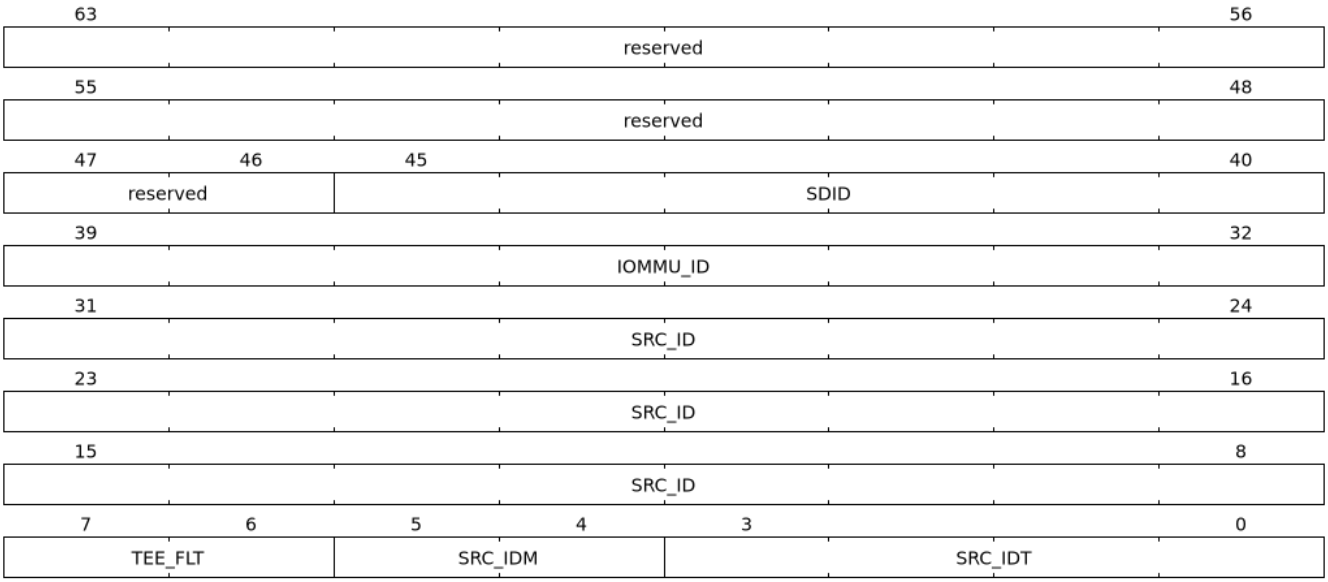


Figure 5. `data1` register layout for `SET_SDCL_ENTRY` operation

The `SRC_IDT` field identifies the type of identifier from the DMA transaction used by this classification rule. The `SRC_IDT` encodings are listed in Table 8.

Table 8. `data1.SRC_IDT` field encodings

`SRC_IDT`	**Description**
0	None. This rule does not match any incoming transaction. All other fields of the `data1` register are ignored.
1	Filter by device ID. The device ID is specified in `SRC_ID` field.
2	Filter by PCIe IDE stream ID and PCIe segment ID. The IDE stream ID is specified in the bits 7:0 of the `SRC_ID` field and the segment ID in bits 15:8 of the `SRC_ID`. The bits 23:16 of the `SRC_ID` field are ignored.
3 - 7	Reserved for future standard use.
8 - 15	Designated for custom use.



In PCIe systems, an originating device can be pinpointed using a unique 16-bit identifier. This identifier is a composite of the PCI bus number (8 bits), device number (5 bits), and function number (3 bits), collectively referred to as the routing identifier or RID. In scenarios where an IOMMU manages multiple hierarchies, there's also an optional segment number, which can be up to 8 bits. Each hierarchy in this context represents a distinct PCI Express I/O interconnect topology. Here, the Configuration Space addresses, which are delineated by the Bus, Device, and Function number tuple, remain distinct. Sometimes, the term Hierarchy is synonymous with Segment. Especially when in Flit Mode, the Segment number can be part of a Function's ID.

The SRC_IDM field can configure SRC_ID matching mode for transactions. The SRC_IDM encodings are listed in Table 9.

Table 9. data1.SRC_IDM field encodings

SRC_IDM	**Description**
0	Reserved for future standard use.
1	TOR. If TOR (Top-Of-Range) is selected, the SRC_ID field forms the top of a range of source IDs. If rule *r*'s SRC_IDM is set to TOR, the rule matches any source ID *s* if: *s* is greater than or equal to SRC_ID of rule *r-1* and is less than the SRC_ID of rule *r*. If *r* is 0, then zero is used as the lower bound. If SRC_ID of rule *r-1* is greater than or equal to that of rule *r* and TOR is selected for rule *r*, then rule *r* does not match any address.
2	Unary. If Unary is selected, then this rule matches if all the bits of the source ID of the transaction match the value configured in the SRC_ID field.
3	NAPOT. If NAPOT is selected, then the rule matches a naturally aligned power-of-two range of source IDs. In this mode, the lower bits of the SRC_ID, up to and including the first low-order zero bit, are masked; the unmasked bits are compared with the corresponding bits in the source ID of the transaction to match.



The following example illustrates the use of SRC_IDM=NAPOT when SRC_IDT is by DEVID and a 24-bit PCIe device_id comprised of the segment, bus, device, and function number is used. In the table below, y acts as a placeholder representing any 1-bit value.

Table 10. SRC_IDM with SRC_IDT set to Filter by device ID

SRC_IDM	SRC_ID	**Comment**
2	yyyyyyyy yyyyyyyy yyyyyyyy	One specific seg:bus:dev:func
3	yyyyyyyy yyyyyyyy yyyyy011	seg:bus:dev - any func
3	yyyyyyyy yyyyyyyy 01111111	seg:bus - any dev:func
3	yyyyyyyy 01111111 11111111	seg - any bus:dev:func

The TEE_FLT field may be used to filter transactions associated with a Trusted Execution Environment (TEE). The encodings for the TEE_FLT field can be found in Table 11.

Table 11. data1.TEE_FLT field encodings

TEE_FLT	**Description**
0	Rule matches both TEE-associated and non-TEE associated transactions.
1	Rule matches TEE-associated transactions.
2	Rule matches transactions that are not TEE associated.
3	Reserved for future standard use.

If filtering transactions associated with a TEE is not supported then the only legal values for TEE_FLT

is 0.



PCIe IDE provides security for transactions from one Port to another. These transactions might be initiated by contexts within the device, such as an SR-IOV virtual function, which are associated with a Trusted Execution Environment (TEE). Within the IDE TLP header, there's a "T" bit that helps differentiate transactions related to a TEE. The PCIe TDISP extended TEE Extension introduces an additional "XT" bit that complements the "T" bit. When both "T" and "XT" bit are 0, the transaction is not related to a TEE. If either the "T" or the "XT" bit are 1 then the transaction is related to a TEE. The TEE_FLT filter can be employed to associate these TEE-related transactions with a different supervisor domain than the transactions not related to TEE. This distinction is made even if both types of transactions are received on the same PCIe IDE stream.

The IOMMU_ID field identifies the instance of the IOMMU that should be used to provide address translation and protection for the transactions matching this rule. When the I/O MPT Checker is not associated with an IOMMU, the IOMMU_ID field is ignored by this operation.

If multiple rules are programmed to match a transaction, the implementation may act based on any one of those matching rules.

If a transaction does not match any of the rules, the I/O Bridge is notified of this condition.

5.7.2. Read the configuration of an SDCL rule - GET_SDCL_ENTRY

The GET_SDCL_ENTRY operation reads the configuration of an SDCL rule identified by RULEID. This operation ignores the contents of the data* registers. Upon success, the configuration of the rule identified by command.RULEID is returned in the data1 register.

The contents of the data1 register are UNSPECIFIED if the operation fails or if it is invoked without a preceding SET_SDCL_ENTRY for the specified RULEID. The contents of the data2 register are UNSPECIFIED, regardless of whether the operation completes successfully.

The interpretation of the command and data1 registers for this operation is identical to that of the SET_SDCL_ENTRY operation.

When the I/O MPT Checker is not associated with an IOMMU, the value held in IOMMU_ID field on successful completion of this operation is UNSPECIFIED.

5.7.3. Set Supervisor Domain Configurations - SET_SDCFG_ENTRY

The SET_SDCFG_ENTRY operation configures MPT and QoS ID parameters for a supervisor domain in the I/O MPT Checker. This operation uses the operands in the command, data1, and data2 registers. The fields of these registers are interpreted as follows.

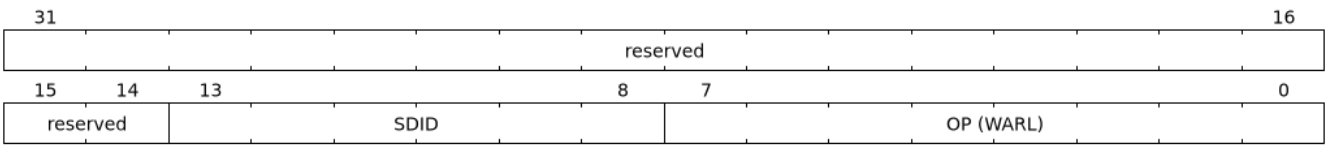


Figure 6. command register layout for SET_SDCFG_ENTRY operation

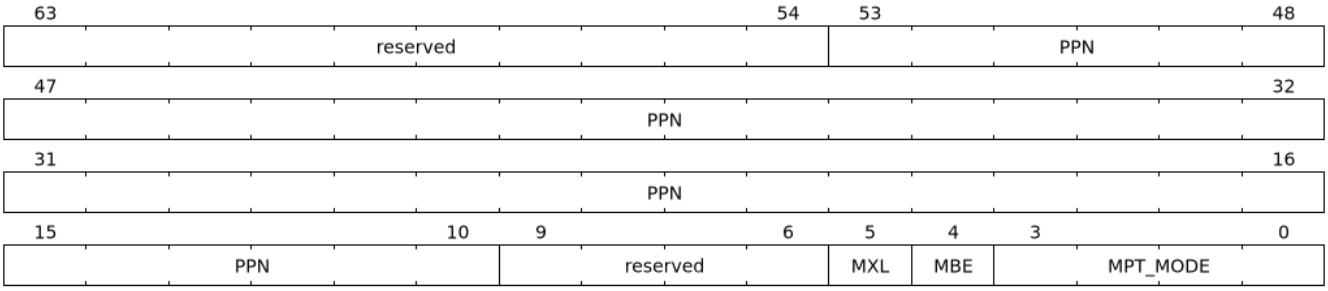


Figure 7. `data1` register layout for `SET_SDCFG_ENTRY` operation

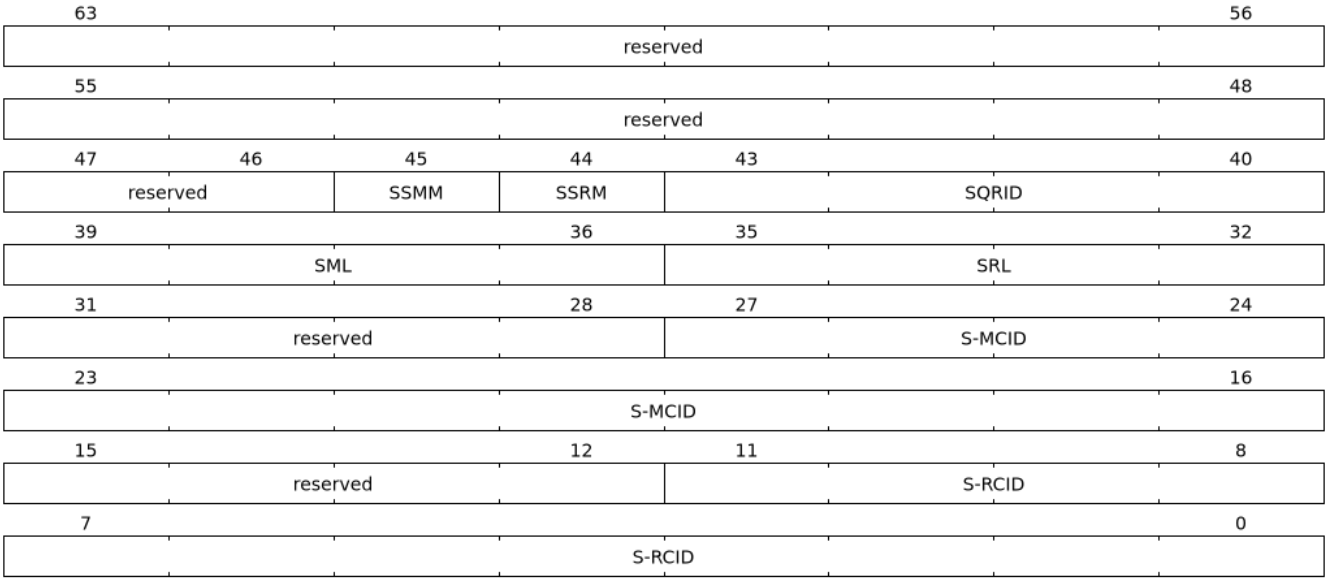


Figure 8. `data2` register layout for `SET_SDCFG_ENTRY` operation

The `MPT_MODE` field identifies the mode of the MPT. It's interpreted as outlined in Table 12 when `MXL` is 0, and as detailed in Table 13 otherwise. The implementation must support MPT modes supported by any of the application processor harts in the system.

Table 12. Encodings of MPT modes when `MXL=0`

Value	Name	Description
0	`Bare`	No page-based memory protection.
1	`Smmpt43`	Page-based memory protection for up to 43-bit physical address spaces.
2	`Smmpt52`	Page-based memory protection for up to 52-bit physical address spaces.
3	`Smmpt64`	Page-based memory protection for up to 64-bit physical address spaces.
4-13	-	`Reserved for future standard use.`
14-15	-	`Designated for custom use.`

Table 13. Encodings of MPT modes when `MXL=1`

Value	Name	Description
0	`Bare`	No page-based memory protection.
1	`Smmpt34`	Page-based memory protection for up to 34-bit physical address spaces.
2-13	-	`Reserved for future standard use.`

Value	Name	Description
14-15	-	Designated for custom use.

The PPN field programs the PPN of the root page of the MPT. When MPT_MODE is Bare, the PPN field must be set to zero.

The MBE field determines the endianness of the access made by I/O MPT Checker to MPT configured in this entry. When this field is 0, the accesses are performed as little-endian accesses and when 1 as big-endian accesses.

The SRL, SML, SSRM, SSMM, S-RCID, and S-MCID fields are used to determine the effective RCID and MCID for device-originated and IOMMU-originated requests, using the RCID-value and MCID-value provided by the IOMMU. These fields shall be implemented if any application processor hart in the system supports the Ssqosid extension. When implemented, they must accept all legal values permitted for the corresponding fields in the srmcfg, mrmcfg, and msdcfg CSRs of any application processor hart in the system. If not implemented, the data2 register value is ignored by this operation.

The determination of the effective RCID and MCID is as follows:

```
SRL_MASK = (1 << SRL) - 1
if SSRM == 0
    effective-RCID = (S-RCID & ~SRL_MASK) | (RCID-value & SRL_MASK)
else
    if ((RCID-value & ~SRL_MASK) | SRL_MASK) != ((1 << RCIDLEN) - 1)
        effective-RCID = RCID-value
    else
        effective-RCID = <unspecified but legal value>
    endif
endif

SML_MASK = (1 << SML) - 1
if SSMM == 0
    effective-MCID = (S-MCID & ~SML_MASK) | (MCID-value & SML_MASK)
else
    if ((MCID-value & ~SML_MASK) | SML_MASK) != ((1 << MCIDLEN) - 1)
        effective-MCID = MCID-value
    else
        effective-MCID = <unspecified but legal value>
    endif
endif
```

The SQRID identifies the QRI for requests originating from the devices and the IOMMU associated with the SD, and accompanies the effective RCID and MCID in the requests made by the device or IOMMU to the QRI.

5.7.4. Get Supervisor Domain Configurations - GET_SDCFG_ENTRY

The GET_SDCFG_ENTRY operation reads the configuration of the supervisor domain identified by SDID from the I/O MPT Checker. This operation ignores the contents of the data* registers. Upon success, the configuration of the supervisor domain identified by command.SDID is returned in the data1 and data2 registers.

The contents of the data1 and data2 registers are UNSPECIFIED if the operation fails or if it is invoked without a preceding SET_SDCFG_ENTRY for the specified SDID.

The interpretation of the command, data1, and data2 registers for this operation is identical to that of the SET_SDCFG_ENTRY operation.

When the I/O MPT Checker does not support associating QoS IDs with requests, the value held in the data2 register on successful completion of this operation is UNSPECIFIED.

5.7.5. MPT Permission Cache Invalidation - MPTINVAL

This operation ensures that stores to an MPT are observed by the I/O MPT Checker before subsequent implicit reads by I/O MPT Checker to the corresponding MPT.

This operation uses the operands in the command register and the data1 register. The contents of the data2 register are ignored by this operation.

For this operation, the fields of the command and data1 registers are interpreted as follows.

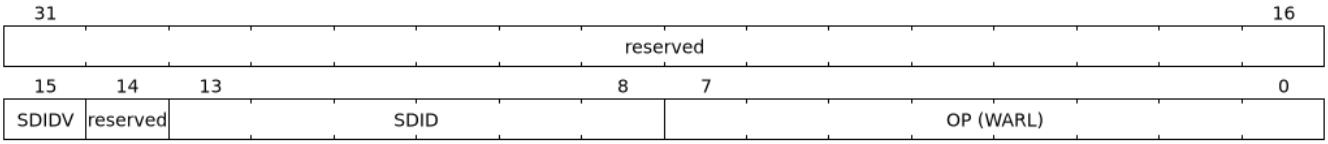


Figure 9. command register layout for MPTINVAL operation

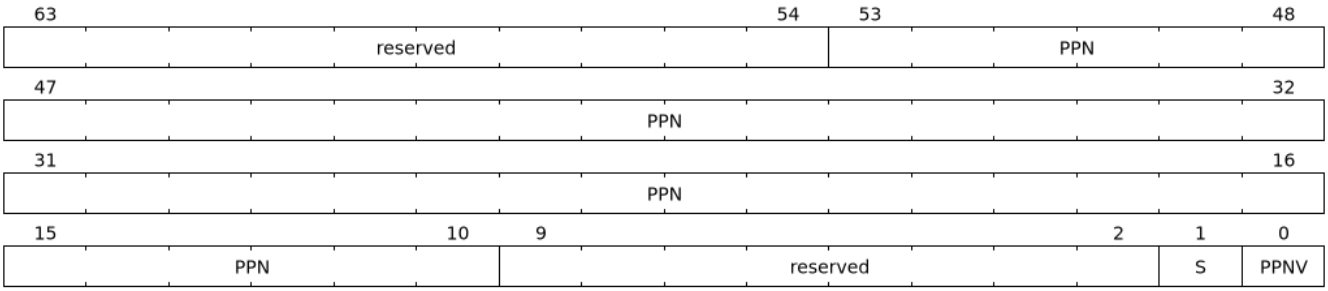


Figure 10. data1 register layout for MPTINVAL operation

The PPNV field indicates if the PPN field is valid and the SDIDV field indicates if the SDID field is valid for the operation. When a field is not valid for an operation, it is ignored by the operation. When the PPNV field is 1, the S field sets the address range size for the MPTINVAL operation. With an S field value of 0, the range size is 4 KiB. But, when the S field has a value of 1, the MPTINVAL operation focuses on a NAPOT range. This range is decided by the low-order bits of the PPN field, going up to the first low-order 0 bit. If the initial low-order 0 bit position of the PPN field is denoted as x, the size of the range is computed as (1 << (13 + x)).

When PPNV is set to 1, if the address range specified by PPN and S is invalid, the operation may or

may not be performed. If the PPNV and S are both set to 1 and all bits of the PPN operand are 1, the behavior of MPTINVAL operation is UNSPECIFIED.

Table 14. MPTINVAL operands and operations

PPNV	SDIDV	Operation
0	0	Invalidates information cached from any MPT for all supervisor domain address spaces.
0	1	Invalidates information cached from the MPT for the address space of the supervisor domain identified by the SDID operand.
1	0	Invalidates information cached from the MPT for the address range in the PPN operand for all supervisor domain address spaces.
1	1	Invalidates information cached from the MPT for the address range in the PPN operand for the supervisor domain address space identified by the SDID operand.



The following example illustrates the use of S field to specify an address range for the MPTINVAL operation. The example shows encoding ranges of up to 8 GiB. Larger ranges may be encoded using the upper address bits (bits 43:22) of the PPN field.

Table 15. Examples of specifying address range sizes using S field

PPN[21:0]	S	Address Range Size
yyyyy yyyyyyyy yyyyyyyy	0	4 KiB
yyyyy yyyyyyyy yyyyyyy0	1	8 KiB
yyyyy yyyyyyy0 11111111	1	2 MiB
yyy01 11111111 11111111	1	1 GiB
01111 11111111 11111111	1	8 GiB



Simpler implementations may ignore the operands of MPTINVAL operation and perform a global invalidation of all information cached from any MPT.

A consequence of this specification is that an implementation may use any information for an address that was valid in the MPT at any time since the most recent MPTINVAL that subsumes that address.

Even if the I/O MPT Checker denies a request, it is allowed to cache the corresponding MPT entry if it is valid.

5.7.6. I/O Fence Operation - IOFENCE

Ensure that all previous read and write requests from devices already processed by I/O MPT Checker are committed to a global ordering point such that they can be observed by all RISC-V harts, IOMMUs, and devices in the system.

The contents of the data1 and data2 registers are ignored by this operation. For this operation, the

fields of the command register are interpreted as follows.

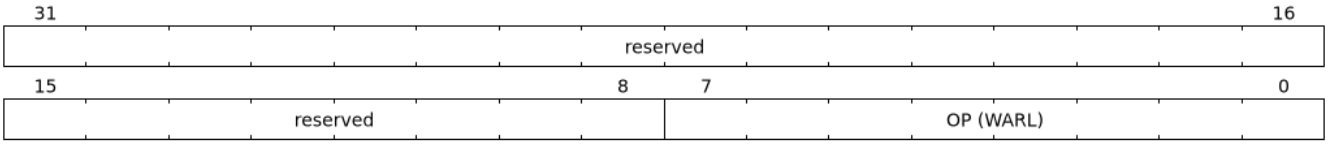


Figure 11. command register layout for IOFENCE operation

If a previously valid SDCL rule, and/or the supervisor domain configurations referenced by a valid SDCL rule, are modified, then the I/O MPT Checker must observe the new configurations within a bounded but UNSPECIFIED time. Software may use the IOFENCE operation after updating the configurations. Successful completion of an IOFENCE operation also guarantees that the I/O MPT Checker has observed all configuration updates made prior to the IOFENCE. If the configuration updates modify the MPT mode and/or PPN, an MPTINVAL operation with PPNV set to 0 must be issued before the IOFENCE.

5.8. Treatment of Device-Side Address Translation Caches

Some devices may participate in the translation process and provide a device-side ATC (DevATC) for their own memory accesses. By providing a DevATC, the device shares the translation caching responsibility and thereby reduces the probability of "thrashing" in the IOATC. The DevATC may be sized by the device to suit its unique performance requirements and may also be used to optimize DMA latency by prefetching translations. Such mechanisms require close cooperation between the device and the IOMMU through a protocol. For PCIe, for example, the Address Translation Services (ATS) protocol may be used by the device to request translations to cache in the DevATC and to synchronize it with updates made by software to address translation data structures.

If the IOBRIDGE and the IOMMUs support the PCIe ATS specification [5], the IOMMU supports two forms of transactions:

- Translated Requests
- PCIe ATS Translation Requests

PCIe ATS Translation Requests are used by the device to obtain a translation for an I/O virtual address (IOVA) specified in the request. These translated addresses may be cached by the device in its DevATC and used as addresses for subsequent Translated Requests.

For a PCIe ATS Translation Request, the IOMMU performs a two-stage address translation to determine the permissions and the size of the translation to be provided in the completion of the request. The IOMMU may be configured to return either the supervisor physical address (SPA) or a guest physical address (GPA) (see T2GPA field in the Device Context, [6]) as the translation of an IOVA in the response.

Translated Requests also invoke the IOMMU to determine if the device is authorized to use these transactions (see EN_ATS field in the Device Context). When the IOMMU is configured to return a GPA in response to a PCIe ATS Translation Request, the IOMMU translates the GPA to an SPA when invoked to process a Translated Request.

If the IOMMU provides an Unsupported Request (UR) or a Completer Abort (CA) as a response to a PCIe ATS Translation Request, the IOBRIDGE forwards that response to the device. If the IOMMU provides a Success response with both R and W permissions set to 0, it indicates that the page requested by the Translation Request is not available, and the IOBRIDGE forwards that response to the device. In these cases, the I/O MPT Checker is not invoked to check the responses.

For all other responses to a PCIe ATS Translation Request, further processing by the IOBRIDGE depends on whether the IOMMU is configured to return an SPA or a GPA in the response.

When the IOMMU is configured to return a GPA as the translated address in response to a PCIe ATS Translation Request, the IOBRIDGE forwards such responses directly to the device. The I/O MPT Checker is not invoked in this case.

When the response to a PCIe ATS Translation Request is an SPA, the IOBRIDGE invokes the I/O MPT Checker on the entire range of translated addresses returned by the IOMMU. If the IOMMU grants read permission to the device, the MPT must also grant read permission; if the IOMMU grants write permission to the device, the MPT must also grant write permission; and if the IOMMU grants permission to execute, the MPT must also grant execute permission for the entire range of translated addresses provided in the PCIe ATS Translation Completion. If the I/O MPT Checker determines that the address is not accessible or that the requested permissions are not present, the IOBRIDGE provides an Unsupported Request (UR) response to the device. If the MPT check succeeds, the translation completion is returned to the device.

For a Translated Request, if the IOMMU provides a UR or CA response, the request is aborted. Otherwise, the IOBRIDGE invokes the I/O MPT Checker on the SPA range provided by the IOMMU. If the MPT check fails, the request is aborted. If the MPT check succeeds, the IOBRIDGE allows the request to access memory.

When protocols other than PCIe ATS are used to support a device-side ATC, the IOBRIDGE shall apply similar rules to enforce memory protection as those defined for PCIe ATS when implementing the I/O MPT Checker functionality.

6. Supervisor Domain Interrupt Assignment

Interrupts that occur during the execution of a supervisor domain cause a trap to the RDSM in M-mode by default.

To increase performance, the RDSM can delegate the handling of certain interrupts, like the timer interrupt or the local performance counter overflow interrupt (LCOFI), to the supervisor domain by setting the appropriate `mideleg` bits. This delegation could necessitate the RDSM to context switch additional state related to such delegated interrupts, including `stimecmp` and the HPM CSRs.

Additionally, the RDSM may assert a *virtual interrupt* to a supervisor domain. Such *virtual interrupts* might be asserted as part of handling a real interrupt, such as a RAS interrupt, or they may have no direct connection to any actual interrupt event.

Some supervisor domains may have devices assigned to them for I/O operations. Devices designed to carry out I/O operations typically signal the completion of an I/O task or an associated error via an external interrupt. These external interrupts might be aggregated and indicated by an interrupt controller linked to the RDSM. The RDSM, in the course of handling an external interrupt, should forward the interrupt to the relevant supervisor domain. The efficiency of handling external interrupts by a supervisor domain can be enhanced if the external interrupts could be directly assigned to the supervisor domain.

To accommodate such supervisor domains, this specification extends the IMSIC and APLIC specified by the RISC-V Advanced Interrupt Architecture (AIA) specification [1] to support multiple *supervisor interrupt domains*, enabling an individual *supervisor interrupt domain* to be associated with each such supervisor domain.

In some systems, a *supervisor interrupt domain* in both an APLIC and an IMSIC may be associated with a supervisor domain, with the supervisor interrupt domain of the APLIC configured to route interrupts as MSIs to the corresponding *supervisor interrupt domain* of the IMSIC.

Interrupt controllers other than APLIC and/or IMSIC that support multiple *supervisor interrupt domains* may also serve a supervisor domain.

If there are N implemented *supervisor interrupt domains*, they are numbered from 0 to N – 1, and the maximum implemented *supervisor interrupt domain number* is N – 1.

6.1. IMSIC Extension for Supervisor Domains

The IMSIC is extended to support multiple *supervisor interrupt domains*, where each *supervisor interrupt domain* may be associated with a supervisor domain. A *supervisor interrupt domain* in an IMSIC consists of:

- A supervisor-level interrupt file.
- Optionally, one or more guest interrupt files.

Each *supervisor interrupt domain* in an IMSIC is identified by a **supervisor interrupt domain number** (`SIDN`).

When all harts are within a single *hart group* as defined by [1], the memory-mapped address of the supervisor-level interrupt file for SIDN `n` and hart `h`, is:

$$\text{Address} = B + n \times 2^I + h \times 2^D$$

Constants `B`, `C`, and `D` in this and later formulas are specified by [1], while `I` is a new constant for supervisor interrupt domains. Constant `I` must be >= `k + D`, where `k` is the number of bits needed to represent any hart number `h`.

If the largest *supervisor interrupt domain* number is $n_{\max}$, then `q` — the number of bits needed to represent any *supervisor interrupt domain* number — is defined as:

$$q = \lceil \log_2(n_{\max} + 1) \rceil$$

Address `B` must be aligned to a 2^{q+I} address boundary. (This automatically makes `B` also aligned to a 2^{k+D} address boundary as required by the existing [1].)

When there are hart groups, the memory-mapped address of the supervisor-level interrupt file for group `g`, SIDN `n`, and hart `h` within the group, is:

$$\text{Address} = g \times 2^E + B + n \times 2^I + h \times 2^D$$

Constant `E` must be >= `max(k + C, q + I)`, but will usually be much larger.

The `Smsdia` extension (Section 6.3) provides the mechanism to select a *supervisor interrupt domain* in an IMSIC as a source of supervisor-level and guest external interrupts for a hart. The interrupt signal from the selected *supervisor interrupt domain's* supervisor-level interrupt file appears as bit `SEIP` in `mip` and `sip`. The interrupt signals from any *guest interrupt files* of the selected supervisor interrupt domain appear as active bits in the hypervisor CSR `hgeip`.

The `siselect` and `sireg` CSRs access the registers of the supervisor-level interrupt file in the selected *supervisor interrupt domain*. If the selected *supervisor interrupt domain* implements *guest interrupt files* then the `vsiselect` and `vsireg` CSRs access the registers of a *guest interrupt file*, of that *supervisor interrupt domain*, as determined by `hstatus.VGEIN`.

6.2. APLIC Extension for Supervisor Domains

The APLIC supports the concept of multiple interrupt domains where each interrupt domain may be associated with a subset of RISC-V harts at one privilege level. To support the Supervisor Domain use cases, the APLIC must support multiple supervisor-level interrupt domains. A supervisor-level interrupt domain in the APLIC may be then associated with a Supervisor Domain by configuring it in *direct delivery mode* or when the hart uses an IMSIC by configuring it in *MSI delivery mode*.

To support supervisor domains in MSI delivery mode, the `smsiaddrcfgh` register is extended with two new parameters called `DXS` (Domain Index Shift) and `DXW` (Domain Index Width) to help determine the addresses to write outgoing MSIs. The extended `smsiaddrcfgh` has the following format:

```
bits 28:24     DXS (WARL)
bits 22:20     LHXS (WARL)
```

```
    bits 18:16     DXW (WARL)
    bits 11:0      High Base PPN (WARL)
```

Let CI be the Child Index from the sourcecfg[i] corresponding to an interrupt source i. The MSI target address for this interrupt source is then computed as follows:

$$n = \text{CI} \,\&\, \left(2^{\text{DXW}} - 1\right)$$

$$\text{MSI address} = (\text{Base PPN} \,|\, (g \ll (\text{HHXS} + 12)) \,|\, (n \ll \text{DXS}) \,|\, (h \ll \text{LHXS}) \,|\, \text{Guest Index}) \ll 12$$

The parameter DXS relates to the constant I specified in section Section 6.1 as: DXS = (I - 12).

Refer to [1] for the detailed definitions of constants HHXS, g, h, LHXS, and Guest Index.

6.3. "Smsdia" : Standard Extension for Supervisor Domain Interrupt Assignment

The Smsdia extension adds supports to select a *supervisor interrupt domain* in an interrupt controller as a source of supervisor-level and guest external interrupts for a hart using the msdcfg CSR.

The RDSM needs a method to be alerted if an external interrupt, whether a supervisor-level external interrupt or guest external interrupt, is pending for any supervisor domains not currently active on a hart. The RDSM could leverage this notification to inform its scheduling decisions. To facilitate this functionality, the Smsdia extension introduces the msideip and msideie CSRs, along with a machine supervisor domain external interrupt (MSDEI).

6.3.1. Supervisor Interrupt Domain Number

The Smsdia extension introduces the *supervisor interrupt domain number* (SIDN) field of the msdcfg Register 3 register.

The SIDN field selects an supervisor interrupt domain among the plurality of *supervisor interrupt domains* associated with a hart as the active supervisor interrupt domain for supervisor-level external interrupt and guest external interrupts.

The SIDN field is a WLRL field that must be able to hold a value between 0 and the maximum implemented *supervisor interrupt domain number*, inclusive. If there is only a single *supervisor interrupt domain* connected to the hart, then SIDN may be read-only zero.

When SIDN is not the number of an implemented *supervisor interrupt domain* then the following rules apply:

- The supervisor-level external interrupt pending signal indicated in mip.SEIP is 0.
- All non-custom values of siselect that access IMSIC registers designate an *inaccessible* register. When siselect holds the number of an *inaccessible* register, attempts from M-mode or HS-mode to access sireg raise an illegal instruction exception.
- Access to CSR stopei raises an illegal instruction exception.

- The `hstatus.vgein` field is read-only zero.
- `hgeip` and `hgeie` are read-only 0.

When the *supervisor interrupt domain* selected by `msdcfg.SIDN` is an implemented, and is an IMSIC, the following rules apply:

- The supervisor-level external interrupt pending signal of supervisor-level interrupt file of the selected *supervisor interrupt domain* is indicated in `mip.SEIP`.
- The `siselect` and `stopei` CSRs operate on the registers of the supervisor-level interrupt file in the selected *supervisor interrupt domain*.
- The guest external interrupt pending signals of the *guest interrupt files* of selected *supervisor interrupt domain* are indicated in the `hgeip` CSR.
- The `hstatus.VGEIN` selects a *guest interrupt file* in the selected supervisor interrupt domain and `vsiselect` and `vstopei` CSRs operate on the registers of the corresponding *guest interrupt file*.

When the *supervisor interrupt domain* selected by `msdcfg.SIDN` is implemented, and is an APLIC, the following rules apply:

- The supervisor-level external interrupt pending signal of the selected APLIC *supervisor interrupt domain* is indicated in `mip.SEIP`.



The `Smsdia` extension provides for directly associating a *supervisor interrupt domain* with up to 64 supervisor domains. The RDSM may emulate interrupt controllers for additional supervisor domains.

To emulate an IMSIC *supervisor interrupt domain*, the RDSM may use the illegal instruction trap facilitated by AIA-added state-enable bits to `mstateen0/mstateen0h` for emulation purposes.

6.3.2. Hypervisor Guest External Interrupt Registers (`hgeip` and `hgeie`)

When the H extension is also implemented, the implementation must internally maintain an array of registers that hold the state of the `hgeip` and `hgeie` CSRs, with one pair of registers per implemented supervisor interrupt domain number. The `hgeip` and `hgeie` CSRs access the element of this array corresponding to the value of `msdcfg.SIDN`.

6.3.3. Machine supervisor interrupt domain external interrupt pending (`msideip/msideiph`)

The `msideip` is a 64-bit read-only machine-mode register, formatted as shown in Figure 12. The CSR number for `msideip` is 0xF4F.

When MXLEN=32, `msideiph` is a 32-bit read-only register which aliases bits 63:32 of `msideip`. When MXLEN=64, `msideiph` does not exist. The CSR number for `msideiph` is 0xF5F.

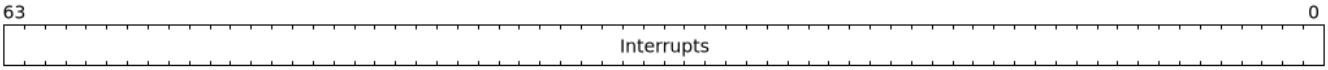


Figure 12. `msideip` register

Each bit *i* in the register summarizes the external interrupts pending in the *supervisor interrupt domain* numbered *i*.

When the *supervisor interrupt domain* identified by *i* is implemented by an APLIC, the bit *i* indicates the state of the supervisor-level external interrupt pending signal provided by the *supervisor interrupt domain* in that APLIC.

When the *supervisor interrupt domain* identified by *i* is implemented by an IMSIC, bit *i* must be set to one if and only if either of the following conditions holds:

- the SEIP signal for *supervisor interrupt domain i* is asserted; or
- the bitwise logical AND of the internal `hgeip` and `hgeie` registers for *supervisor interrupt domain i* is nonzero (i.e., the value that SGEIP would have for that domain).

The summary of external interrupts pending in a *supervisor interrupt domain* is visible in the `msideip` register even when `msdcfg.SIDN` is not the valid number of an implemented *supervisor interrupt domain*.

6.3.4. Machine supervisor interrupt domain external interrupt enable (`msideie/msideieh`)

The `msideie` is a 64-bit read-write machine-mode register, formatted as shown in Figure 13. The CSR number for `msideie` is 0x74F.

When MXLEN=32, `msideieh` is a 32-bit read-write machine-mode register which aliases bits 63:32 of `msideie`. When MXLEN=64, `msideieh` does not exist. The CSR number for `msideieh` is 0x75F.

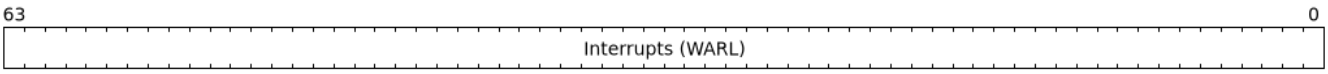


Figure 13. `msideie` register

The `msideie` CSR selects the subset of *supervisor interrupt domains* that cause a machine supervisor domain external interrupt (`MSDEI`). The enable bits in `msideie` do not affect the supervisor-level external interrupt and guest external interrupt pending signals from the *supervisor interrupt domain* selected by `msdcfg.SIDN`.

6.3.5. Machine and Supervisor Interrupt registers (`mip/mie` and `sip/sie`)

The `Smsdia` extension introduces the machine supervisor domain external interrupt (`MSDEI`). This interrupt is assigned to bit 14 in the `mip`, `mie`, `sip`, and `sie` registers. The bit 14 in `mip` and `sip` is called `MSDEIP` and the same bit in `mie` and `sie` is called `MSDEIE`. The `mideleg` register controls the delegation of `MSDEI` to S-mode. This interrupt cannot be delegated to VS-mode and bit 14 of `hideleg` is read-only zero.

The `mip.MSDEIP` bit read-only and is set to 1 if the bitwise logical AND of CSRs `msideip` and `msideie` is nonzero in any bit. The `mie.MSDEIE` bit is writable.

When bit 14 of `mideleg` is zero, `sip.MSDEIP` and `sie.MSDEIE` are read-only zeros. Else, `sip.MSDEIP` and `sie.MSDEIE` are aliases of `mip.MSDEIP` and `mie.MSDEIE`.

Multiple simultaneous interrupts destined for different privilege modes are handled in decreasing order of destined privilege mode. For the base Privileged ISA's major interrupts (numbers 0-15), multiple simultaneous interrupts destined for the same privilege mode are handled in the following decreasing default priority order: MEI, MSI, MTI, MSDEI, SEI, SSI, STI, SGEI, VSEI, VSSI, VSTI, LCOFI.



The RDSM may use the machine supervisor interrupt domain external interrupt to determine if a supervisor domain has become ready to run since it was last descheduled. When a supervisor domain that has a supervisor domain interrupt controller directly assigned to it, the RDSM updates the `msdcfg.SIDN` to select that supervisor interrupt domain and may clear the bit corresponding to that supervisor interrupt domain in `msideie` prior to resuming execution of the supervisor domain.

The RDSM may delegate `MSDEI` to a supervisor domain that may be entrusted by the RDSM to get notified about supervisor domain external interrupts pending for one or more other supervisor domains. Typically, this use case involves a single supervisor domain that is trusted by the RDSM to receive such notifications. The delegation supports optimizing the exit sequence from such supervisor domain by enabling such supervisor domains to voluntarily yield execution in response to pending interrupts for the other supervisor domains.

6.4. "Smgeien/Ssgeien" : Standard Extension for Machine Guest External Interrupt Enables

The `Smgeien` extension enables access to a subset of the guest interrupt files within a *supervisor interrupt domain* in an IMSIC to a supervisor domain. The `Smgeien` extension depends on the `Smsdia` extension.

The `Smgeien` extension depends on the `Smaia` and the H extension. When `Smgeien` is implemented, the bit 9 of `mvien` must be writable.

This extension introduces an MXLEN read-write WARL register named `mgeien`, formatted as shown in Figure 14 for RV64 and in Figure 15 for RV32. The CSR number is 0x74D.

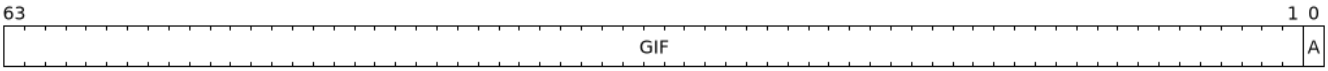


Figure 14. `mgeien` register for RV64

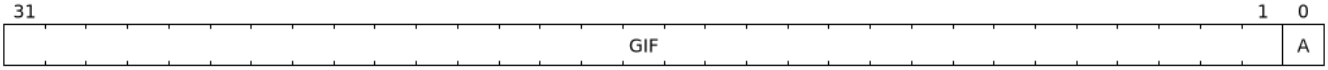


Figure 15. `mgeien` register for RV32



The RDSM may use bit 9 of `mvien` to control access to the supervisor-level interrupt file in the *supervisor interrupt domain* selected by `msdcfg.SIDN`.

Each bit of the `GIF` field when set to 1 enables access to corresponding *guest interrupt file* in the *supervisor interrupt domain* selected by `msdcfg.SIDN`.

The number of writable bits in mgeien is determined by the maximum GEILEN value across all supervisor interrupt domains, and this maximum is referred to as MGEILEN. If MGEILEN is nonzero, bits MGEILEN:1 shall be writable in mgeien. All other bit positions in the GIF field shall be read-only zeros.

For a supervisor-level environment, extension Ssgeien is essentially the same as Smgeien except excluding the machine-level CSRs and behavior not directly visible to supervisor level.

If the *supervisor interrupt domain* selected by msdcfg.SIDN is implemented then:

- Bits GEILEN:1 of hgeip are read-only aliases of the same bits in the internal hgeip register selected by msdcfg.SIDN if the bit at the same position in mgeien is 1. All other bits of hgeip are zero.
- Bits GEILEN:1 of hgeie are writable aliases of the same bits in the internal hgeie register selected by msdcfg.SIDN if the bit at the same position in mgeien is 1. All other bits of hgeie are read-only zero.
- If hstatus.VGEIN is not 0 and the bit selected by VGEIN in mgeien is 1 then the vsiselect, vsireg, and vstopei CSRs operate on the registers of the *guest interrupt file* selected by VGEIN. If VGEIN is 0 or the the bit selected by VGEIN in mgeien is 0 then the vsiselect, vsireg, and vstopei CSRs operate as-if the VGEIN is the number of an unimplemented guest external interrupt.

The mgeien.A bit, when set to 1, enables msideip to accumulate all external interrupts for the supervisor interrupt domain; otherwise, msideip accumulates only the subset of guest external interrupts determined by mgeien.GIF. When mgeien.A is 0, the SEIP for the supervisor interrupt domain is accumulated in msideip if it is not delegated to the supervisor domain.

Each bit *i* of msideip, corresponding to an implemented *supervisor interrupt domain*, must be set to one if and only if the relevant interrupt source is active.

Specifically:

- If *i* equals msdcfg.SIDN, the bit is set to one if and only if either:
  - mip.SEIP is 1 and bit 9 (for SEI) of mideleg is zero; or
  - mip.SEIP is 1 and mgeien.A is 1; or
  - The bitwise logical AND of the following three components of the *supervisor interrupt domain* is nonzero:
    - the *internal* hgeip,
    - the *internal* hgeie, and
    - the *logical complement* of mgeien if mgeien.A is 0; otherwise, a value of all 1s.

The number of bits implemented in hgeip and hgeie is referred to as GEILEN. When GEILEN is nonzero, the Privileged Specification requires that the least significant bits, excluding bit 0, be implemented first. Hence, if GEILEN is nonzero, bits GEILEN:1 shall be writable in hgeie, and all other bit positions shall be read-only zeros in both hgeip and hgeie. When the Ssgeien extension is implemented, this rule does not apply, and:

- The guest external interrupts and their enables may not be contiguous in hgeip and hgeie, respectively.

- The supervisor-level and guest interrupt files may not be laid out contiguously in memory.
- The supervisor-level interrupt file may not be available to a supervisor domain even when guest interrupt files are available.



The `Smgeien` extension is intended for use cases involving supervisor domains that share the guest interrupt files in a *supervisor interrupt domain*.

Typically, one of the supervisor domains participating in the scheme acts as a donor domain that initially has access to all the interrupt files. Such a donor supervisor domain may then donate one or more interrupt files—usually _guest interrupt files—to a recipient supervisor domain, either temporarily or permanently, by invoking SBI functions provided by the RDSM. Once the interrupt files have been made accessible to the recipient domain by the RDSM, the donor domain is not expected to access them.

The memory locations of the accessible guest interrupt files, along with their guest external interrupt numbers, are communicated to recipient supervisor domains by the RDSM through software mechanisms such as SBI.

7. Supervisor Domain External Trace and Debug Allow Controls

This chapter describes two extensions `Smsdedbga` and `Smsdetrca` that enable the RDSM to enforce isolation of supervisor domains when external debug and external trace mechanisms are being utilized.

`Smsdedbga` defines the `SEDA` bit in the `msdcfg` CSR that specifies if a supervisor domain is allowed to be externally-debugged.

Similarly, `Smsdetrca` defines the `SETA` bit in the `msdcfg` CSR that specifies if a supervisor domain is allowed to be externally-traced [7].

External trace refers to tracing the execution of software running on a separate target RISC-V platform through a trace control and data transport mechanism that provides external access to the trace encoder controls [8].

These two extensions only affect debug and trace orchestrated by an external agent - self-hosted debug (and trace) are managed directly by the RDSM.

This chapter refers to the Debug specification [7], Trace specification [8], and the External debug security extension [9] for the description and behavior of controls outside the scope of this specification, but which interact with controls specified in this specification when one or more supervisor domains are used.



The values of `msdcfg.SEDA` and `msdcfg.SETA` shall be obtained from the configuration information defined for the supervisor domain and reflected in the attestation report for that domain and its workloads. The RDSM shall preserve and restore this configuration state across context switches using secure memory.

These configurations allow external debug and trace but do not enable them. If external debug and/or trace is allowed for a supervisor doamin, an external debugger may enable the external debug and/or trace for that supervisor domain.

7.1. `Smsdedbga`: External Debug allowed control for Supervisor Domain

When M-mode external debug is enabled, all supervisor domains may also be debugged by an external debugger irrespective of the configuration held in `msdcfg.SEDA`.



Whether external debug of M-mode is permitted is determined by the hardware root of trust (RoT) ([10]) of the RDSM and is outside the scope of this specification. When such debug is enabled, the RDSM itself is under debug, and all supervisor domains managed by the RDSM are implicitly under debug as well.

When M-mode external debug is disabled, whether execution at privilege modes less than `M` may be

debugged by an external debugger depends on the configuration held in `msdcfg.SEDA`, as described below:

If `msdcfg.SEDA` is 0, the hart shall disallow all external debug access using mechanisms defined in the RISC-V Debug Specification [7].

If `msdcfg.SEDA` is 1, the hart shall permit external debug access to privilege modes less than M.

To enforce the above controls specified by this extension, the following functional and security requirements [9] must be met:

- External debug of M-mode can be enabled only by the HW RoT of the RDSM.
- The RDSM must be able to context switch all triggers even when `d-mode` = 1. This functional change allows the RDSM to remain in control of external debug for supervisor domains when it is not under external debug itself.

7.2. `Smsdetrca`: External Trace allowed control for Supervisor Domain

When M-mode external trace is enabled, all supervisor domains activity may also be traced by an external trace tool irrespective of the configuration held in `msdcfg.SETA`.



Whether external tracing of M-mode is permitted is determined by the hardware root of trust (RoT) ([10]) of the RDSM and is outside the scope of this specification. When such tracing is enabled, the RDSM itself is under debug, and all supervisor domains managed by the RDSM are implicitly under debug as well.

When M-mode external trace is disabled, whether execution at privilege modes less than M-mode may be traced by an external trace tool depends on the configuration held in `msdcfg.SETA`, as described below:

If `msdcfg.SETA` is 0, the hart shall disallow all external trace.

If `msdcfg.SETA` is 1, the hart shall permit external trace of privilege modes less than M, subject to the following requirements:

- When M-mode external tracing is disabled, external tracing shall be stopped upon:
  - Entry into Debug Mode.
  - Entry into M-mode.

To enforce these controls, the following functional and security requirements [9] shall be met:

- External trace of M-mode shall be enabled only by the hardware root of trust (HW RoT) of the RDSM.

8. Capacity and Bandwidth QoS Register Interface (CBQRI) for Supervisor Domains

Quality of Service (QoS) is defined as the minimal end-to-end performance that is guaranteed in advance by a service level agreement (SLA) to a workload. A workload may be a single application, a group of applications, a virtual machine, a group of virtual machines, or a combination of those.

8.1. Introduction to CBQRI and Ssqosid

The RISC-V Capacity and Bandwidth QoS Register Interface (CBQRI) [11] specification specifies mechanisms to a) associate QoS identifiers with requests made by workloads to resources controlled by capacity and bandwidth controllers, b) memory-mapped register interface for capacity allocation and capacity usage monitoring in capacity controllers in shared resources such as shared caches, c) memory-mapped register interface for bandwidth allocation and bandwidth usage monitoring in bandwidth controllers in shared resources such as memory controllers.

The Ssqosid extension [12] provides a read/write S/HS-mode register (`srmcfg`) to configure QoS Identifiers to be used with requests made by the hart to shared resources. The CBQRI specification specifies the IOMMU extension to associate QoS Identifiers with requests made by devices to shared resources.

8.2. Security objective

With supervisor domains, the resource accesses from an SD or the RDSM must not be observable by entities that are not within their TCB using the resource usage monitors. Similarly, the resource allocations for an SD or the RDSM must not be influenced by entities outside their TCB.

To support this security objective, the following capabilities are required:

- **Multiple QoS register interfaces (QRIs) in the capacity and bandwidth controllers within the SoC.**
  - The RDSM may allocate a QRI exclusively to an SD or may mediate access to a QRI among multiple SDs. A non-ISA supervisor domain QRI extension is specified in Section 8.5 to provide this capability.
- **Associating a QRI ID (QRID) with requests originating from SDs and RDSM.**
  - The QRID, along with the `RCID` and `AT`, is used to identify the resource allocation configurations in a capacity or bandwidth controller. The QRID, along with the `MCID`, is used to identify the ID of the counter used to monitor resource usage in a capacity or bandwidth controller.
  - The Smsdqosid Section 8.4 extension is specified to enable partitioning the RCID and MCID associated with a QRI among the SDs sharing a QRI.
  - The Smqosid extension Section 8.3 is specified to configure QoS identifiers for requests made by a hart with effective privilege mode equal to M.

8.3. Smqosid - Machine-Mode Quality-of-Service Identifiers

The Smqosid introduces a read/write M-mode register (`mrmcfg`) to configure QoS identifiers for requests made by a hart with effective privilege mode equal to M.

The `mrmcfg` register is an MXLEN-wide read/write machine-mode register used to configure a Resource Control ID (`RCID`), Monitoring Counter ID (`MCID`), and a QoS Register Interface ID (`QRID`). The `QRID`, `RCID`, and `MCID` are WARL fields. The register is formatted as shown in Figure 16 when MXLEN=64 and Figure 17 when MXLEN=32. The CSR number for `mrmcfg` is 0x381.

The `QRID`, `RCID`, and `MCID` values accompany each request made with effective privilege mode equal to M by the hart to shared resource controllers. The `QRID` and `RCID` values are used to determine the resource allocations (e.g., cache occupancy limits, memory bandwidth limits, etc.) to enforce. The `QRID` and `MCID` values are used to identify a counter to monitor resource usage.



A suggested reset value of 0 for the `RCID` and `QRID` fields within the `mrmcfg` register aligns with the default operational behavior of resource controllers, which is to associate all capacity and resources with `RCID=0`. This setup ensures a seamless initial state that conforms to expected default resource allocation policies. The reset value for the `MCID` field, being potentially implementation-defined, does not directly impact core functionality and may vary based on specific system requirements.

The `QRID` is typically selected once at startup by the RDSM to designate the QRI utilized by the RDSM itself.

When Smqosid is implemented, the configurations in the `srmcfg` register introduced by Ssqosid extension only apply to requests made with effective privilege modes less than M.

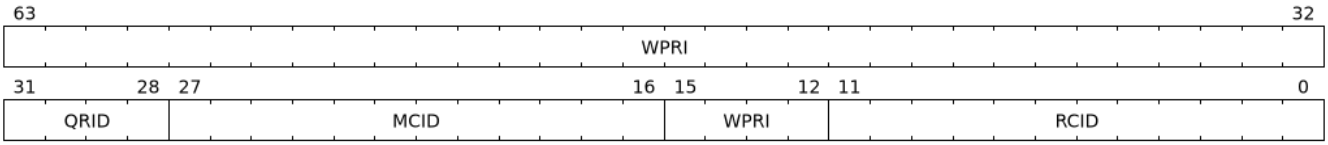


Figure 16. Machine Resource Management Configuration (`mrmcfg`) register for MXLEN=64

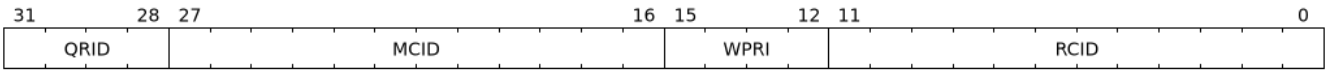


Figure 17. Machine Resource Management Configuration (`mrmcfg`) register for MXLEN=32

8.4. Smsdqosid - Supervisor Domain Quality-of-Service Identifiers

The Smsdqosid extension enables the scenario where two or more supervisor domains share a common set of capacity and bandwidth controllers. In this setup, access to the QRI of these controllers is restricted to RDSM, which then provides mediated access for these supervisor domains to configure capacity or bandwidth allocation and read the corresponding monitoring

counters. A common QRID is associated with requests from these domains, and the RDSM allocates sets of RCID and MCID for each domain's use. Supervisor domains are then restricted to selecting a RCID and MCID in srmcfg from the allocated range. Although the RDSM could limit access to srmcfg to prevent direct domain configuration, enabling domains to program srmcfg directly is preferred for performance reasons.

To facilitate delegation of srmcfg to the SD, the Smsdqosid extension introduces a MXLEN-wide read/write machine-mode register called mnrmcfg. The register is formatted as shown in Figure 18 for MXLEN=64 and Figure 19 for MXLEN=32. The CSR number for mnrmcfg is 0x781.

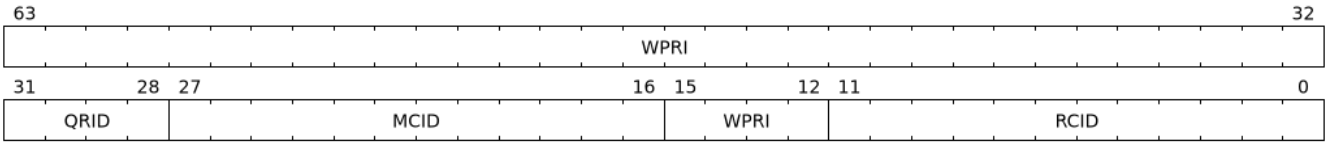


Figure 18. Machine Next-Privilege Resource Management Configuration (mnrmcfg) register for MXLEN=64

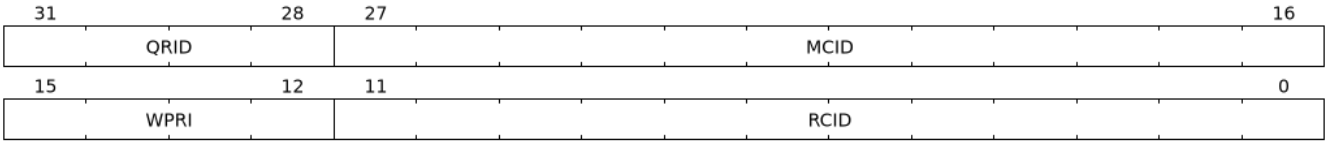


Figure 19. Machine Next-Privilege Resource Management Configuration (mnrmcfg) register for MXLEN=32

Additionally, the extension introduces the SRL, SML, SSRM, and SSMM fields in the msdcfg Register 3 register.

The SRL (supervisor-RCID-length) is a WARL field with legal values from 0 to RCIDLEN, where RCIDLEN is the number of implemented bits of the RCID field of srmcfg. The least-significant bits of RCID are implemented first. That is, if RCIDLEN is nonzero, then RCID[RCIDLEN-1:0] is writable.

The SML (supervisor-MCID-length) is a WARL field with legal values from 0 to MCIDLEN, where MCIDLEN is the number of implemented bits of the MCID field of srmcfg. The least-significant bits of MCID are implemented first. That is, if MCIDLEN is nonzero, then MCID[MCIDLEN-1:0] is writable.

The supervisor-srmcfg-rcid-mode (SSRM) and SRL fields in the msdcfg determine the legal values for the srmcfg.RCID field. When SSRM is 0, the legal values are between 0 and $2^{SRL} - 1$. When SSRM is 1, the legal values are between 0 and $2^{RCIDLEN} - 2^{SRL} - 1$, and in this mode SRL must satisfy $0 \leq SRL < RCIDLEN$.

The supervisor-srmcfg-mcid-mode (SSMM) and SML fields determine the legal values for the srmcfg.MCID field. When SSMM is 0, the legal values are between 0 and $2^{SML} - 1$. When SSMM is 1, the legal values are between 0 and $2^{MCIDLEN} - 2^{SML} - 1$, and in this mode SML must satisfy $0 \leq SML < MCIDLEN$.

These ranges constrain only the values that may be written to and held in srmcfg.RCID and srmcfg.MCID. The effective RCID and MCID values that accompany requests to the shared resource controllers may differ when SSRM or SSMM is 0, as defined by the following equations.

The mnrmcfg.RCID and SSRM determine the value of RCID that accompanies requests made with effective privilege mode less than M by the hart to shared resource controllers as follows:

```
SRL_MASK = (1 << SRL) - 1
RCID = (SSRM == 0) ? ((mnrmcfg.RCID & ~SRL_MASK) | (srmcfg.RCID & SRL_MASK)) :
```

```
                    srmcfg.RCID
```

The mnrmcfg.MCID and SSMM determine the value of MCID that accompanies requests made with effective privilege mode less than M by the hart to shared resource controllers as follows:

```
SML_MASK = (1 << SML) - 1
MCID = (SSMM == 0) ? ((mnrmcfg.MCID & ~SML_MASK) | (srmcfg.MCID & SML_MASK)) :
                    srmcfg.MCID
```

These RCID and MCID values along with mnrmcfg.QRID accompany each request made with effective privilege mode less than M by the hart to shared resource controllers. The QRID and RCID values are used to determine the resource allocations (e.g., cache occupancy limits, memory bandwidth limits, etc.) to enforce. The QRID and MCID values are used to identify a counter to monitor resource usage.



Consider a QRI that supports 32 RCIDs and is mediated by RDSM between two SDs. The RDSM may allocate 24 RCIDs to the first SD and 8 RCIDs to the second SD. The SRL and SSRM configurations used by the RDSM to support this use case are as follows:

- The RDSM configures SRL to 3 and SSRM to 1 for first SD. This allows the first SD to program RCID values between 0 and 23 in srmcfg. The value of RCID that accompanies the requests from this SD is same as the value in srmcfg.
- The RDSM configures SRL to 3, SSRM to 0, and mnrmcfg.RCID to 24 for the second SD. This allows the second SD to program RCID values 0 through 7 in srmcfg. The value of RCID that accompanies the requests from this SD are between 24 and 31 depending on the value held in srmcfg.RCID.

The RDSM is expected to configure the mnrmcfg fields and the msdcfg fields such that each SD is associated with a disjoint range of values for RCID and MCID.



The mnrmcfg.QRID may be updated during the process of switching SDs, to denote the QRI allocated to that specific SD by the RDSM.

When a QRI is shared among multiple supervisor domains, the RDSM must mediate access to the memory-mapped QoS register interface provided by the capacity and bandwidth controllers. This mediation may be implemented by the RDSM either by emulation of a virtual QoS register interface—intercepting and emulating loads and stores to these registers—or by providing a supervisor binary interface (SBI) for QoS configuration. Programming of the QoS controllers is typically performed at job startup and is infrequent compared to the switching of RCID and/or MCID values in srmcfg during a context switch.

When a supervisor domain is allocated a dedicated QRI, the RDSM may permit direct access from that domain to the QRI's memory-mapped register interface. In this case, the RDSM should set SSRM and SSMM to 0 and configure SRL and SML to match the supported RCIDLEN and MCIDLEN values of the QRI. The RCID and MCID values programmed into the srmcfg CSR accompany requests issued by the

supervisor domain. Each QRI is associated with a distinct RCID and MCID namespace.

8.5. Supervisor Domain QoS Register interfaces (QRI)

Capacity and bandwidth controllers that support supervisor domains provide one or more memory-mapped QoS register interfaces (QRI). A QRI may be made exclusively accessible to a supervisor domain or the RDSM may mediate access to the QRI using an SBI. The RDSM can control access to the QRI from supervisor domains using MPT and/or PMP. The RDSM controls access to the QRI from devices using IOMPT and/or IOPMP.

The number of `RCID` and `MCID` supported by the controllers for each QRI need not be identical. For maximal flexibility in allocation of `RCID` and `MCID` values, it is recommended that the number of `RCID` and `MCID` supported for a given QRID be identical in all capacity and bandwidth controllers in the system.

The capacity and bandwidth controllers use the configurations that were established for the `RCID` and `AT` in the request through the QRI corresponding to the QRID in the request. Likewise the counters to count resource usage are selected using `MCID` and the QRID in the request and a QRI can be used to access counters associated with the corresponding QRID.

When a controller supports only a single QRI, machine mode is required to mediate access to that QRI. The RDSM may implement this mediation either by emulating a virtual QoS register interface—intercepting and emulating loads and stores to those registers—or by providing a supervisor binary interface (SBI) for QoS configuration.

When a capacity or bandwidth controller supports multiple QRIs, all resources in the controller are, by default, available for allocation through any QRI. The controller may optionally support reservation of resources for exclusive use by a specific QRI. When such reservation is supported, capacity or bandwidth may be reserved for allocation only by that QRI. Support for resource reservation enables effective partitioning of shared resources among supervisor domains (SDs) that share the controller.

When multiple QRIs are implemented, one of them must be designated for use by machine mode. This QRI is known as the *machine-mode QRI* (MQRI).

The MQRI supports a new operation, `CONFIG_QRI_LIMIT` (`OP=4`), which enables partitioning of shared resources—such as cache capacity or memory bandwidth—among the QRIs by specifying limits on the amount of each resource that may be allocated by the corresponding QRI. If this operation is requested on any QRI other than the MQRI, it shall fail with `STATUS=2`.

The MQRI may not support any `RCID` or `MCID`, and when these are unsupported, its primary role is to configure resource limits for other QRIs managed by the controller. When `RCID` and/or `MCID` are supported, the MQRI may be used exclusively by machine mode or may be shared with one or more supervisor domains.

Bits 31:28 of the `cc_alloc_ctl` and the `bc_alloc_ctl` registers of an MQRI are defined as the `QRID` field. This field is reserved in all other QRIs of the controller. It is used by the `CONFIG_QRI_LIMIT` (`OP=4`) operation to specify the QRID to which the limit applies and is ignored by all other operations.

To reserve capacity for a QRI, the RDSM may use the CONFIG_QRI_LIMIT operation through the MQRI, with the QRID field identifying the target QRI. The capacity to be reserved is specified using the cc_block_mask register. The AT and RCID fields of the cc_alloc_ctl register are ignored by this operation.

The mask specified in cc_block_mask must contain a contiguous run of 1s, and an implementation may require the mask to reserve at least one capacity block; otherwise, the operation shall fail with STATUS=5.

The number of 1 bits in the cc_block_mask value specifies the number of capacity blocks that may be allocated using the identified QRI.

On successful completion, the NCBLKS field in the cc_capabilities register of the target QRI shall reflect the updated number of allocatable capacity blocks.



The CONFIG_QRI_LIMIT operation is typically a one-time operation used by the RDSM to configure the capacity limit for a QRI before making it accessible to an SD. The SD may then allocate capacity for RCIDs within the limit established by the RDSM.



Let's consider a cache with NCBLKS=8. In this example, this cache supports two QRIs with QRID of 0 and 1. The CONFIG_QRI_LIMIT operation is used to reserve two capacity blocks numbered 7 and 6 for use by QRID=0. The CONFIG_QRI_LIMIT operation is used to reserve six capacity blocks numbered 0 through 5 for use by QRID=1. The SD that uses the QRID=0 is thus limited to selecting a 2 bit capacity block mask where the bit 0 of the mask maps to capacity block 6 and bit 1 to capacity block 7. The SD that uses QRID=1 is limited to selecting a 6 bit capacity block mask where the mask bits 0 through 5 map to the correspondingly numbered capacity blocks. Both SDs in this example, have configured RCID=5 with 1 capacity block for requests with access-type AT=0. The effective capacity block allocation in the controller is as follows:

	7	6	5	4	3	2	1	0
QRID=0, RCID=5, AT=0	0	1	0	0	0	0	0	0
QRID=1, RCID=5, AT=0	0	0	0	0	0	1	0	0

To reserve bandwidth for a QRI, the RDSM may use the CONFIG_QRI_LIMIT operation through the MQRI, with the QRID field identifying the target QRI. The amount of bandwidth to reserve is specified using the Rbwb and Mweight fields of the bc_bw_alloc register. The AT and RCID fields of the bc_alloc_ctl register are ignored by this operation. Similarly, the sharedAT and useShared fields of the bc_bw_alloc register are also ignored.

When the specified Mweight is nonzero, it represents a proportional share of non-reserved or unused bandwidth that may be used by RCIDs associated with the QRI. When Mweight is zero, the value in Rbwb serves as a hard limit, and RCIDs associated with the QRI are not eligible to use any non-reserved or unused bandwidth.

On successful completion of the operation, the bc_capabilities.MRBWB field of the target QRI shall reflect the value specified in Rbwb.

When the Mweight established for a QRI is nonzero, a two-level weighted sharing of unused or non-reserved bandwidth occurs. In such cases, the unused bandwidth is apportioned among all QRIs with Mweight != 0 that are contending for it, based on their relative Mweight values. Specifically, the proportion of unused bandwidth allocated to QRID **q** is determined by dividing its Mweight by the sum of the Mweight values across all QRIs contending for unused bandwidth, as shown in equation (1).

This proportion P_q of the unused bandwidth is then further distributed among the RCIDs associated with that QRI using the weights configured for the RCIDs.

$$P_q = \frac{Mweight_q}{\sum_{q=1}^{n} Mweight_q} \qquad (1)$$

Consider a bandwidth controller that supports two QRIs. For brevity, this example controller does not support bandwidth allocation by access-type AT. In this example, the QRID=0 has been configured with Rbwb of 100 bandwidth units and QRID=1 has been configured with Rbwb of 50 bandwidth units. The Mweight configured for the two QRIs is 16, i.e., they equally share unused bandwidth.

Each QRI in this example is used to configure bandwidth limits for RCID=5 and RCID=6 where each RCID has been allocated 10 units of reserved bandwidth and configured with weights 50 and 25, respectively.

With this configuration RCID=5 receives 2/3 of the unused bandwidth made available to the QRI and RCID=6 receives 1/3 of the unused bandwidth made available to the QRI when they both contend for the unused bandwidth.

The effective configurations in the bandwidth controller are as follows:

	RCID Rbwb	RCID Mweight	QRI Rbwb	QRI Mweight
QRID=0, RCID=5	10	50	100	16
QRID=0, RCID=6	10	25	100	16
QRID=1, RCID=5	10	50	50	16
QRID=1, RCID=6	10	25	50	16

The bandwidth enforcement is typically work-conserving, meaning that it allows unused bandwidth to be used by QRIs enabled to use it even if they have consumed their Rbwb.

When contending for unused bandwidth, the weighted share is typically computed among the QRIs that are actively generating requests in that accounting interval and have a non-zero weight programmed.

Typical use cases may require only a small number of QRIs (e.g., 1 to 4). In confidential computing scenarios, two QRIs may suffice: one designated for use by the non-confidential host OS or hypervisor, and another retained under RDSM

control to mediate access on behalf of a confidential OS or hypervisor. The number of RCIDs and MCIDs supported by the two QRIs does not need to be identical.

All QoS controllers in the SoC are strongly encouraged to implement the same number of QRIs. For each QRID, the corresponding QRI should support the same number of RCIDs and MCIDs across all controllers. For example, if QRID=1 supports 8 RCIDs and 4 MCIDs in one controller, QRID=1 should support the same in every controller. QRID=2 may support a different number of RCIDs and MCIDs than QRID=1, but it should do so consistently across all controllers. Furthermore, the QRID assigned to the machine-mode QRI should be the same across all controllers. This uniformity facilitates efficient and predictable allocation of QRIs and RCIDs/MCIDs to supervisor domains (SDs).

Bibliography

[1] “RISC-V Advanced Interrupt Architecture.” [Online]. Available: github.com/riscv/riscv-aia.

[2] “RISC-V Confidential VM Extension.” [Online]. Available: github.com/riscv-non-isa/riscv-ap-tee.

[3] “The RISC-V Instruction Set Manual Vol. II: Privileged Architecture.” [Online]. Available: github.com/riscv/riscv-isa-manual.

[4] “Common Terminology for Confidential Computing.” [Online]. Available: confidentialcomputing.io/wp-content/uploads/sites/10/2023/03/Common-Terminology-for-Confidential-Computing.pdf.

[5] “PCI-SIG specifications.” [Online]. Available: pcisig.com/specifications.

[6] “RISC-V IOMMU specification.” [Online]. Available: github.com/riscv-non-isa/riscv-iommu/releases.

[7] “RISC-V Debug Specification.” [Online]. Available: github.com/riscv/riscv-debug-spec.

[8] “RISC-V Efficient Trace for RISC-V, v2.0.2, March 5, 2024.” [Online]. Available: github.com/riscv-non-isa/riscv-trace-spec/releases/download/v2.0.2/riscv-trace-spec-asciidoc.pdf.

[9] “RISC-V External Debug Security, v0.0, March 26, 2024.” [Online]. Available: github.com/riscv-non-isa/riscv-external-debug-security/blob/main/external-debug-security.pdf.

[10] “TCG Glossary.” [Online]. Available: trustedcomputinggroup.org/resource/tcg-glossary/.

[11] “RISC-V Capacity and Bandwidth Controller QoS Register Interface.” [Online]. Available: github.com/riscv-non-isa/riscv-cbqri.

[12] “RISC-V Quality-of-Service (QoS) Identifiers.” [Online]. Available: github.com/riscv/riscv-ssqosid.